



NOWITNESS

Penetration Test Report

Lace Mobile App Security Assessment

Date January 26, 2026
Client Input Output (IO)
Project Lace Mobile App
Version 1.0.0-alpha

Contents

Disclosure	1
Disclaimer	2
Scope	3
Assessment Overview	4
Assessment Components	5
Executive Summary	6
Code Base	7
Severity Classification	8
Finding Severity Ratings	9
Findings	10
[C 501] Authentication Bypass (UI-Gated Secret Access)	12
[H 401] Encrypted Seed Stored in Unprotected File Storage	15
[H 402] Android Auto Backup Enabled	16
[H 403] BUILD-01: Dev/Test Feature Flags Enabled in Production Defaults	18
[H 404] BUILD-02: Android Build Injects mavenLocal() Repository	20
[H 405] BUILD-06: Android Backup/Restore Configuration Needs Verification	22
[M 301] No Screenshot/Screen Recording Prevention	24
[M 302] Unsafe Onboarding Flows	28
[M 303] Missing Root and Jailbreak Detection	32
[M 304] CRYPTO-01: PBKDF2 Insufficient Iterations (EMIP-3 Encryption)	35
[M 305] DEP-02: Vulnerable Dev Dependencies	38
[M 306] NUMERIC-01: Bitcoin Satoshi Calculations Use JavaScript Number Type	41
[M 307] NUMERIC-02: Number/BigInt Mixing in Bitcoin Input Selection	45
[M 308] BUILD-03: iOS Build Uses Remote Podspec Without Integrity Verification .	49
[M 309] BUILD-04: Sentry Initialized with Debug Mode and Full Sampling	51
[L 201] No SSL/Certificate Pinning	53
[L 202] Wallet Renaming Without Re-authentication	58
[L 203] DEP-01: Vulnerable NPM Dependencies	59
[L 204] NUMERIC-03: rawToBigInt parseInt Precision Loss	63
[L 205] BUILD-05: API Keys Bundled in Client Application	66
[L 206] BUILD-07: iOS Export Compliance Declaration May Be Inaccurate	68
[L 207] BUILD-08: Dev Artifacts Exclusion from Release Builds Needs Verification .	70
[L 208] BUILD-09: Feature Flag Override Mechanism in Production Builds	72
[I 101] Native Apollo Sync API Inconsistencies (Latent Risk)	74
[I 102] Non-functional Address Book	78

[I 103]	Missing Balance Privacy Feature	80
[I 104]	Transaction and Wallet Transfer via QR and File Import	81
[I 105]	Sensitive Cryptographic Material Exposure Risk	83

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Input Output (IO)** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this penetration test was limited to the Lace Mobile App as deployed in the provided test environment and did not include the underlying blockchain, backend services, host device or operating system, or any external dApps.

Scope

In Scope

Mobile Application

- Key store management
- Transaction signing flows
- Password control mechanisms
- Biometric authentication security
- Recovery phrase handling
- User onboarding flows

Data & Storage

- Secure enclave / Keychain / Keystore
- Local storage and caching
- Memory handling and zeroization

Client-side API Handling

- Request/response data exposure
- Sensitive data in transit (client POV)
- Error handling and info leakage

Supply Chain

- Dependency analysis
- Third-party library vulnerabilities

Out of Scope

- Third-party dApps (unless whitelisted)
- Backend infrastructure / server-side
- Social engineering attacks
- Physical device attacks
- App store distribution security

Assessment Overview

From **December 16, 2025** to **January 26, 2026**, **Input Output (IO)** engaged No Witness Labs to conduct a comprehensive security assessment of the **Lace Mobile App** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured four-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Review** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

Assessment Components

Manual review

Our manual review covers the Lace Mobile App (iOS and Android) and its client-side interactions with supporting services/APIs. It focuses on attack surfaces and failure modes defined in the agreed scope, including but not limited to:

- Key store management and protected secret handling (Secure Enclave/Keychain/Keystore)
- Authentication controls (password/PIN, biometrics) and lock/unlock behavior
- Recovery phrase handling and onboarding flows
- Transaction signing flows and integrity of user-visible review versus signed intent
- Local storage/caching behavior, memory handling, and zeroization
- Client-side API handling (request/response exposure, transport security, error/info leakage)
- Telemetry/logging exposure and data minimization (client POV)
- Build/release hardening and dependency/supply-chain review

Executive Summary

Project Overview

Lace Mobile App is a non-custodial wallet application for iOS and Android. The app is responsible for generating or restoring wallet key material on the user's device, protecting that material with platform security controls, and enabling users to review and sign transactions for supported networks (including Cardano and Bitcoin).

As a client-side wallet, Lace Mobile's security depends primarily on correct enforcement of authentication gates (password/PIN and biometrics), safe handling of recovery phrases and sensitive secrets, robust transaction-signing flows, and secure local persistence and telemetry practices. This assessment evaluated those mobile-specific security properties and the surrounding build and dependency posture for the reviewed codebase.

Code Base

Repository

<https://github.com/input-output-hk/lace-platform.git>

Tag

lace-mobile@1.0.0-alpha

Commit

2784ab108e98f5406adfe2bab5e26dd1e3bb792b

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Status
	5	Critical	1
	4	High	5
	3	Medium	9
	2	Low	8
	1	Informational	5
Total Findings			28

Findings

ID	Title	Severity	Status
501	Authentication Bypass (UI-Gated Secret Access)	Critical	Acknowledged
401	Encrypted Seed Stored in Unprotected File Storage	High	Acknowledged
402	Android Auto Backup Enabled	High	Acknowledged
403	BUILD-01: Dev/Test Feature Flags Enabled in Production Defaults	High	Acknowledged
404	BUILD-02: Android Build Injects mavenLocal() Repository	High	Acknowledged
405	BUILD-06: Android Backup/Restore Configuration Needs Verification	High	Acknowledged
301	No Screenshot/Screen Recording Prevention	Medium	Acknowledged
302	Unsafe Onboarding Flows	Medium	Acknowledged
303	Missing Root and Jailbreak Detection	Medium	Acknowledged
304	CRYPTO-01: PBKDF2 Insufficient Iterations (EMIP-3 Encryption)	Medium	Acknowledged
305	DEP-02: Vulnerable Dev Dependencies	Medium	Acknowledged
306	NUMERIC-01: Bitcoin Satoshi Calculations Use JavaScript Number Type	Medium	Acknowledged
307	NUMERIC-02: Number/BigInt Mixing in Bitcoin Input Selection	Medium	Acknowledged
308	BUILD-03: iOS Build Uses Remote Podspec Without Integrity Verification	Medium	Acknowledged

ID	Title	Severity	Status
309	BUILD-04: Sentry Initialized with Debug Mode and Full Sampling	Medium	Acknowledged
201	No SSL/Certificate Pinning	Low	Acknowledged
202	Wallet Renaming Without Re-authentication	Low	Acknowledged
203	DEP-01: Vulnerable NPM Dependencies	Low	Acknowledged
204	NUMERIC-03: rawToBigInt parseInt Precision Loss	Low	Acknowledged
205	BUILD-05: API Keys Bundled in Client Application	Low	Acknowledged
206	BUILD-07: iOS Export Compliance Declaration May Be Inaccurate	Low	Acknowledged
207	BUILD-08: Dev Artifacts Exclusion from Release Builds Needs Verification	Low	Acknowledged
208	BUILD-09: Feature Flag Override Mechanism in Production Builds	Low	Acknowledged
101	Native Apollo Sync API Inconsistencies (Latent Risk)	Informational	Acknowledged
102	Non-functional Address Book	Informational	Acknowledged
103	Missing Balance Privacy Feature	Informational	Acknowledged
104	Transaction and Wallet Transfer via QR and File Import	Informational	Acknowledged
105	Sensitive Cryptographic Material Exposure Risk	Informational	Acknowledged

[C 501] Authentication Bypass (UI-Gated Secret Access)

	Level	Severity	Status
	5	Critical	Acknowledged

Category: Authentication & Secret Storage

Affected Files:

- packages/contract/onboarding-v2/src/store/side-effects.ts
- packages/contract/authentication-prompt/src/store/side-effects/biometrics-side-effects.ts
- packages/contract/authentication-prompt/src/authenticators/password/secure-store-password-manager.ts

Description

The wallet password (`passwordHex`) is stored in and retrieved from `expo-secure-store` with `requireAuthentication: false`. This means the operating system's hardware-backed authentication gate is not enforced. The authentication prompt users see is purely a UI/UX layer (`expo-local-authentication`), not a hardware-enforced access control.

What `requireAuthentication` controls:

Setting	iOS (Keychain)	Android (Keystore)
<code>true</code>	Uses <code>biometryCurrentSet</code> access control; OS prompts for FaceID/TouchID before releasing data	Equivalent to <code>setUserAuthenticationRequired(true)</code> ; key material locked in TEE until auth
<code>false</code>	Data accessible to any code in app context without prompt	Key decryption does not require user authentication

Current Implementation

All three critical operations explicitly pass `false`:

1. Creating the password (onboarding):

```
1 // packages/contract/onboarding-v2/src/store/side-effects.ts:150
2 const autoGeneratedPassword = passwordManager.createPassword(false);
```

typescript

2. Storing an existing password (onboarding fallback):

```
1 // packages/contract/onboarding-v2/src/store/side-effects.ts:183
2 passwordManager.setPassword(AuthSecret(ByteArray(passwordBytes)), false);
```

typescript

3. Retrieving the password after "auth success":

```
1 // packages/contract/authentication-prompt/src/store/side-effects/
  biometrics-side-effects.ts:37
2 const storedPassword = passwordManager.getPassword(false);
```

typescript

The “UI-Gate” Illusion

The app does show a authentication prompt via `expo-local-authentication`. However, this is a separate API from the Keychain/Keystore access control. The flow is:

1. JS code calls `LocalAuthentication.authenticateAsync()` → OS shows FaceID prompt
2. If user's face matches, JS receives `{ success: true }`
3. JS code then calls `secureStore.getItem('passwordHex', { requireAuthentication: false })`
4. OS immediately returns the secret — no prompt, no hardware check

An attacker who can:

- Hook into the JS runtime (Frida, debugger)
- Run code on a rooted/jailbroken device
- Exploit any code execution vulnerability

...can skip step 1-2 and directly call step 3-4 to obtain `passwordHex`.

Attack Chain

1. Obtain `passwordHex`: On a compromised device, call `SecureStore.getItem('passwordHex')` directly (no auth prompt will appear).
2. Obtain `encryptedRecoveryPhrase`: Extract from MMKV storage (see finding 02 — MMKV is unencrypted).
3. Decrypt mnemonic: Use `emip3decrypt(encryptedRecoveryPhrase, passwordHex)` with the same libraries the app uses.
4. Full wallet compromise: The recovery phrase gives complete control over all funds.

Proof of Concept Approach

1. Build a debug version of the app or attach to a running instance
2. After completing onboarding, call:

```
1 import * as SecureStore from 'expo-secure-store';javascript
2 const password = SecureStore.getItem('passwordHex', { requireAuthentication:
  false });
3 console.log(password); // Will print without any biometric prompt
```

3. Compare with:

```
1 const password = SecureStore.getItem('passwordHex',javascript
  { requireAuthentication: true });
2 // This WILL trigger FaceID/TouchID prompt
```

Affected Code

- `packages/contract/onboarding-v2/src/store/side-effects.ts`
 - (line: 150): `createPassword(false)`
 - (line: 183): `setPassword(..., false)`

- `packages/contract/authentication-prompt/src/store/side-effects/biometrics-side-effects.ts`
 - (line: 37): `getPassword(false)`
- `packages/contract/authentication-prompt/src/authenticators/password/secure-store-password-manager.ts`
 - (line: 15): Default param is `false`

References

- [Expo SecureStore Documentation](#)
- [iOS Keychain Access Control](#)
- [Android Keystore User Authentication](#)

Recommendation

Change all callsites to use `requireAuthentication: true`:

```

1 // packages/contract/onboarding-v2/src/store/side-effects.ts
2 - const autoGeneratedPassword = passwordManager.createPassword(false);
3 + const autoGeneratedPassword = passwordManager.createPassword(true);
4
5 - passwordManager.setPassword(AuthSecret(ByteArray(passwordBytes)), false);
6 + passwordManager.setPassword(AuthSecret(ByteArray(passwordBytes)), true);
7
8 // packages/contract/authentication-prompt/src/store/side-effects/biometrics-side-effects.ts
9 - const storedPassword = passwordManager.getPassword(false);
10 + const storedPassword = passwordManager.getPassword(true);
  
```

Additional considerations:

- Remove the separate `expo-local-authentication` call — the OS will handle the prompt when you try to access the Keychain item with `requireAuthentication: true`
- Handle the case where biometrics are unavailable (fallback to password entry)
- Test on both iOS and Android as behavior differs slightly

[H 401] Encrypted Seed Stored in Unprotected File Storage

	Level	Severity	Status
	4	High	Acknowledged

Category: Storage Security

Affected File: `packages/module/storage-react-native-async/src/storage-api-adapter.ts`

Description

The `encryptedRecoveryPhrase` is persisted via Redux/MMKV, which writes to plaintext files on the device filesystem:

```
1 const storage = new MMKV(); // No encryption typescript
```

While the seed is EMIP-3 encrypted, storing it in file-based storage exposes it to backup extraction (Android `adb backup`) and filesystem access on compromised devices. This enables offline brute-force attempts against the encryption.

Note: This becomes Critical when combined with finding F501, as the attacker then has both the ciphertext and the key.

Recommendation

Move `encryptedRecoveryPhrase` from MMKV to SecureStore. To avoid double biometric prompts, use different access controls:

Item	Storage	<code>requireAuthentication</code>
<code>passwordHex</code>	SecureStore	<code>true</code>
<code>encryptedRecoveryPhrase</code>	SecureStore	<code>false</code>

```
1 // Store encrypted phrase (no auth required - it's already encrypted) typescript
2 await SecureStore.setItemAsync(
3   `wallet:${walletId}:encryptedPhrase`,
4   encryptedRecoveryPhrase,
5   { requireAuthentication: false }
6 );
```

This keeps the UX to a single biometric prompt (for `passwordHex`) while moving the encrypted seed out of file-based storage. SecureStore is excluded from Android backups and has better OS-level protection than MMKV files.

Additionally, set `android:allowBackup="false"` in the manifest as defense in depth.

[H 402] Android Auto Backup Enabled

	Level	Severity	Status
	4	High	Acknowledged

Category: Platform Security / Data Backup

Affected Files:

- `apps/lace-mobile/app.json`
- `packages/module/storage-react-native-async/src/storage-api-adapter.ts`

Description

The Android application does not explicitly disable the `android:allowBackup` attribute in the manifest configuration, which defaults to `true` in Expo applications. This configuration allows the Android Auto Backup system to include application data in backups to Google Drive.

Data Included in Backups

MMKV storage is initialized without encryption:

```
1 // packages/module/storage-react-native-async/src/storage-api-adapter.ts:5
2 const storage = new MMKV();
```

The following data persisted via Redux is included in system backups:

- `encryptedRecoveryPhrase` (EMIP-3 encrypted seed phrase)
- Wallet metadata (names, order, account structure)
- Token balances and metadata
- Transaction history
- Network settings and preferences

Data excluded from backups:

- `expo-secure-store` entries (Keystore/Keychain items are automatically excluded per platform specifications)
- User authentication credentials (`passwordHex`)

Potential Attack Scenarios

1. ADB Backup Extraction:

```
1 adb backup -f backup.ab io.lace.mobilewallet
```

The backup archive contains MMKV files with persisted application state.

2. Compromised Google Account: Unauthorized access to a user's Google account provides access to backup data stored in Google Drive.
3. Device Restoration: Application data may be restored to a different device during the Android restore process.

4. Offline Cryptanalysis: The `encryptedRecoveryPhrase` extracted from backups may be subject to offline cryptographic attacks.

References

- [OWASP Mobile Top 10 2016 - M1: Improper Platform Usage](#)
- [OWASP Mobile Top 10 2016 - M2: Insecure Data Storage](#)
- [Android Auto Backup Documentation](#)
- [Expo SecureStore Documentation](#)

Recommendation

1. Disable Android Auto Backup

Modify `apps/lace-mobile/app.json` to explicitly set `allowBackup` to `false`:

```

1  {
2    "expo": {
3      "android": {
4        "adaptiveIcon": {
5          "foregroundImage": "./assets/Lace-android-adaptive-icon.png",
6          "backgroundColor": "#FFFFFF"
7        },
8        "package": "io.lace.mobilewallet",
9        "permissions": [
10         "android.permission.USE_BIOMETRIC",
11         "android.permission.USE_FINGERPRINT"
12       ],
13       "allowBackup": false
14     }
15   }
16 }
```

This configuration will generate `android:allowBackup="false"` in the `AndroidManifest.xml` during the build process.

2. Verification Steps

After implementing the configuration change, verify the manifest contains the correct attribute:

```

1  eas build --platform android --profile development
2
3  unzip -p app-release.apk AndroidManifest.xml | xmllint --format - | grep allowBackup
```

Expected output should contain `android:allowBackup="false"`.

[H 403] BUILD-01: Dev/Test Feature Flags Enabled in Production Defaults

	Level	Severity	Status
	4	High	Acknowledged

Category: Build Configuration / Debug Surfaces

Description

The file `apps/lace-mobile/src/app/feature-flags.ts` is explicitly documented as being used for production builds, but it currently enables development/testing flags by default:

```
1 export const defaultFeatureFlags: FeatureFlag[] = [
2   { key: FeatureFlagKey('ANALYTICS_POSTHOG') },
3   { key: FeatureFlagKey('FEATURES_DEV') },
4   // ...
5   { key: FeatureFlagKey('TEST_API') },
6   // ...
7 ];
```

On a fresh install, `apps/lace-mobile/src/app/util/load-modules.ts` falls back to these compile-time defaults when the feature flag document is missing (`defaultIfEmpty(defaultFeatureFlags)`), so `FEATURES_DEV` and `TEST_API` are active before any user action.

Because:

- `@lace-module/feature-dev` and `@lace-module/test-api` are part of the mobile module registry (`apps/lace-mobile/src/app/util/all-modules.ts`), and
- their `feature.willLoad(...)` checks do not restrict by environment,

they can be selected and loaded in `NODE_ENV=production` whenever these flags are present.

What is actually exposed (by default flags)

TEST_API (@lace-module/test-api)

On `lace-mobile`, this module defines `addons.loadInitializeMobileView` and currently reuses the extension initializer (`packages/module/test-api/src/initialize-extension-view.ts`). When loaded, it attaches a privileged API surface to the global `window`:

- State access
 - `window.store`: Redux store (full read/write via dispatch)
- Wallet mutation
 - `window.addWallet(wallet)`: injects a wallet entity into the store
 - `window.removeWallet(walletId)`: removes wallet + associated accounts
- Sensitive reads (via selectors)
 - `window.getWallets()`

- `window.getAddresses(blockchain)`
- `window.getTokens()`
- `window.getAnalyticsUser()`
- `window.getColorScheme()`
- UI mutation
 - `window.setColorScheme(colorScheme)`
 - `window.setLanguage(language)`
- Test/automation helpers
 - `window.waitForState(predicate)`
 - `window.reloadExtension()`
 - `window.toggleReactScan()` (uses `localStorage` / `window.location.reload()`, so may be non-functional or error on native)

FEATURES_DEV (@lace-module/feature-dev)

- In `lace-mobile` specifically, `@lace-module/feature-dev` does not currently expose `window.setFeatureFlags(...)` by default, because it only defines `addons.loadInitializeExtensionView` (no `addons.loadInitializeMobileView`).
- However, the module does wire up a `featureFlags$` observable and listens for the `featureDev/setFeatureFlags` action to update/persist feature flags. With `TEST_API` exposing `window.store`, any code with store access can dispatch a `featureDev/setFeatureFlags` action and persistently replace feature flags (which can change which modules/features load).

Severity rationale

This is rated **High** because the default production flags load a module (`TEST_API`) that exposes full store access and wallet mutation helpers. Even though exploitation typically requires access to the app JS runtime (e.g., via instrumentation or another code-execution primitive), this is a highly privileged surface that should never be enabled by default in production builds.

Recommendation

1. Remove `FEATURES_DEV` and `TEST_API` from `defaultFeatureFlags` (production defaults). Require explicit opt-in via `feature-flags.override.ts` (dev-only) or a controlled remote-flag system.
2. Add defense-in-depth by gating these modules themselves:
 - In `@lace-module/feature-dev` and `@lace-module/test-api`, make `feature.willLoad(...)` return `false` for `environment === 'production'`.
3. Add CI checks for production builds that fail if:
 - `apps/lace-mobile/src/app/feature-flags.ts` contains `FEATURES_DEV` / `TEST_API`
 - the built bundle contains `@lace-module/test-api` / `@lace-module/feature-dev` exports or known debug globals.

[H 404] BUILD-02: Android Build Injects mavenLocal() Repository

Level	Severity	Status
 4	High	Acknowledged

Category: Build Configuration / Supply Chain

Affected Files:

- apps/lace-mobile/expo-build-plugin/src/android.ts
- apps/lace-mobile/expo-build-plugin/src/gradle-dependencies.json
- ci/android/build-android.sh
- packages/apollo-native/android/installApolloMaven.sh

Description

The Expo build plugin injects `mavenLocal()` into the Android project's `allprojects.repositories` block. In the current implementation this is injected at the top of the repositories list, so it can take precedence over remote repositories.

In the current CI release build flow, Android builds explicitly install an Apollo AAR into `~/.m2/repository`:

- ci/android/build-android.sh → packages/apollo-native/android/installApolloMaven.sh .
- That makes `mavenLocal()` part of the production build path, not just developer convenience.

Important context (why this exists)

We currently rely on a locally-published Maven artifact to ship the native Apollo module:

- CI publishes `org.hyperledger.identity.apollo:apollo-android:1.6.0` into `~/.m2/repository`
- The Expo build plugin adds:
 - `implementation('org.hyperledger.identity.apollo:apollo-android:1.6.0')`

So, some form of “local Maven repository” is required unless Apollo is moved to a different distribution mechanism (e.g. private artifact registry or vendored repo).

What the issue is (and isn't)

- This is not “using a local Apollo artifact is wrong”.
- The risk comes from enabling global, unscoped `mavenLocal()` (especially when it is first in the repo list), which allows any dependency coordinate to be satisfied from `~/.m2/repository` if present.

An attacker who can poison the local Maven cache (e.g., compromised CI runner, cache poisoning, or a compromised developer machine producing a release) could place a malicious artifact with matching coordinates. Gradle would then resolve the malicious artifact from `~/.m2/repository` instead of a trusted remote source.

This reduces build hermeticity and creates a supply-chain injection vector that bypasses remote repository integrity checks.

Recommendation

1. Prefer a dedicated workspace-local Maven repository directory over `~/.m2/repository`:
 - Publish Apollo into a repo directory inside the workspace (e.g., `packages/apollo-native/android/maven-repo/`)
 - Point Gradle at that repo via `maven { url uri(...) }` and remove `mavenLocal()`
2. If `mavenLocal()` must remain, constrain blast radius:
 - Use Gradle repository content filtering so only `org.hyperledger.identus.apollo` can resolve from `mavenLocal()`
 - Ensure `mavenLocal()` is last in the repository list (so it cannot override deps that are available in trusted remotes)
 - Run CI builds with a clean/isolated `HOME` or clear `~/.m2/repository` before installing Apollo
3. Enable Gradle dependency verification (`gradle/verification-metadata.xml`) and fail CI on unexpected checksum changes.
4. Add CI checks to verify the generated Gradle files do not contain `mavenLocal()` (or that it is content-filtered to only the intended group) for release configurations.

[H 405] BUILD-06: Android Backup/Restore Configuration Needs Verification

	Level	Severity	Status
	4	High	Acknowledged

Category: Build Configuration / Data Backup

Affected Files:

- `android/app/src/main/AndroidManifest.xml`
- `apps/lace-mobile/app.json`
- `packages/module/storage-react-native-async/src/storage-api-adapter.ts`

Description

Android's backup mechanism can automatically backup app data to Google Drive, including potentially sensitive data stored in the app's private directory. For a wallet application, this could expose:

- MMKV database files containing sensitive persisted state (e.g. wallet metadata and potentially `encryptedRecoveryPhrase` depending on what is persisted)
- Cached authentication state
- User preferences and wallet metadata

The generated `AndroidManifest.xml` must be verified to contain proper backup restrictions. If `android:allowBackup` is not explicitly set to `false`, the default is `true` on Android 6.0+.

In this repo:

- `apps/lace-mobile/app.json` does not currently declare an explicit Android backup policy (no `android.allowBackup` setting).
- The mobile storage adapter uses `react-native-mmkv` without an `encryptionKey` (`new MMKV()`), so any backed-up MMKV files are readable once extracted.

This finding still benefits from artifact verification (generated manifest), but the current configuration provides no explicit assurance that backup is disabled.

Recommendation

1. Verify current state by inspecting the generated `AndroidManifest.xml` after running `npm expo prebuild`:

```
1 grep -E "allowBackup|fullBackupContent" android/app/src/main/AndroidManifest.xml
```

bash

2. Disable backup for release builds (preferred for wallet apps):
 - Set `android.allowBackup` to `false` in `apps/lace-mobile/app.json`, or
 - Add an Expo config plugin to set `android:allowBackup="false"` and disable Auto Backup (`android:fullBackupContent="false"`) for the `application` node.
3. Alternative: Selective backup exclusion using `backup_rules.xml`:

```
1 <full-backup-content>
2   <exclude domain="sharedpref" path="."/>
3   <exclude domain="database" path="."/>
4   <exclude domain="file" path="mmkv"/>
5 </full-backup-content>
```

xml

4. Add CI verification to check the manifest in release builds:

```
1 if grep -q 'android:allowBackup="true"' android/app/src/main/
  AndroidManifest.xml; then
2   echo "ERROR: Backup must be disabled for wallet app"
3   exit 1
4 fi
```

bash

5. Defense-in-depth: ensure any sensitive persisted data is encrypted at rest (e.g. MMKV encryptionKey) even when backup is disabled.

[M 301] No Screenshot/Screen Recording Prevention

	Level	Severity	Status
	3	Medium	Acknowledged

Category: UI Security / Data Exposure

Affected Components: App-wide (sensitive routes)

Description

Sensitive screens within the Lace Mobile application, including recovery phrase display and signing prompts, are not protected against screenshots or screen recording. This allows sensitive wallet data to be captured and potentially exfiltrated through various attack vectors.

Affected screens

The following screens display or handle sensitive information without screenshot/recording protection:

1. Recovery phrase display — Shows 12/24 word mnemonic during wallet creation or phrase reveal
2. Recovery phrase input — User enters mnemonic words during wallet restoration
3. Transaction signing prompts — Shows transaction details including amounts and addresses
4. Receive address display — Shows wallet addresses that could be used for targeted attacks

Attack vectors

1. Screen capture malware — Malicious apps with screen recording permissions can capture sensitive screens (common in accessibility-based malware on Android)
2. Accidental cloud backup — Users may screenshot their recovery phrase “for backup”, which automatically syncs to iCloud/Google Photos; cloud account compromise exposes mnemonic
3. Screen sharing attacks — User unknowingly shares screen during support scam; remote access tools capture sensitive screens

Current state

No implementation of screenshot prevention was found in the codebase. A search for `FLAG_SECURE`, `WindowManager`, `setSecure`, or screenshot-related APIs returned no results in the mobile app code.

Recommendation

Android Implementation

Use `FLAG_SECURE` on sensitive activities/fragments:

```
1 // In sensitive Activity or Fragment kotlin
2 override fun onCreate(savedInstanceState: Bundle?) {
3     super.onCreate(savedInstanceState)
4     window.setFlags(
5         WindowManager.LayoutParams.FLAG_SECURE,
6         WindowManager.LayoutParams.FLAG_SECURE
7     )
8 }
```

For React Native, create a native module to toggle secure mode:

```
1 // SecureScreenModule.kt kotlin
2 @ReactMethod
3 fun enableSecureMode() {
4     currentActivity?.runOnUiThread {
5         currentActivity?.window?.setFlags(
6             WindowManager.LayoutParams.FLAG_SECURE,
7             WindowManager.LayoutParams.FLAG_SECURE
8         )
9     }
10 }
11
12 @ReactMethod
13 fun disableSecureMode() {
14     currentActivity?.runOnUiThread {
15         currentActivity?.window?.clearFlags(
16             WindowManager.LayoutParams.FLAG_SECURE
17         )
18     }
19 }
```

iOS Implementation

iOS doesn't have a direct equivalent to `FLAG_SECURE`, but several approaches exist:

1. Screenshot/recording detection with user notification:

```
1 // Show pop-up message when screenshot/recording detected (similar to MetaMask) swift
2 NotificationCenter.default.addObserver(
3     forName: UIApplication.userDidTakeScreenshotNotification,
4     object: nil,
5     queue: .main
```

```

6  ) { _ in
7      self.showSecurityWarning(message: "Screenshots are disabled for security")
8  }
9
10 // Screen recording detection (iOS 11+)
11 if UIScreen.main.isCaptured {
12     self.showSecurityWarning(message: "Screen recording detected")
13 }
14
15 NotificationCenter.default.addObserver(
16     forName: UIScreen.capturedDidChangeNotification,
17     object: nil,
18     queue: .main
19 ) { _ in
20     if UIScreen.main.isCaptured {
21         self.showSecurityWarning(message: "Screen recording detected")
22     } else {
23         self.hideSecurityWarning()
24     }
25 }
26
27 func showSecurityWarning(message: String) {
28     // Display modal or overlay with warning message
29     // Optionally blur sensitive content behind the message
30 }

```

2. Blur overlay on app backgrounding (optional):

```

1  // AppDelegate.swift swift
2  func applicationWillResignActive(_ application: UIApplication) {
3      let blurEffect = UIBlurEffect(style: .light)
4      let blurView = UIVisualEffectView(effect: blurEffect)
5      blurView.frame = window?.frame ?? UIScreen.main.bounds
6      blurView.tag = 999
7      window?.addSubview(blurView)
8  }
9
10 func applicationDidBecomeActive(_ application: UIApplication) {
11     window?.viewWithTag(999)?.removeFromSuperview()
12 }

```

React Native Integration

Create a cross-platform hook for sensitive screens:

```
1 // useSecureScreen.ts typescript
2 import { useEffect } from 'react';
3 import { Platform, NativeModules } from 'react-native';
4
5 const { SecureScreenModule } = NativeModules;
6
7 export const useSecureScreen = (enabled: boolean = true) => {
8   useEffect(() => {
9     if (!enabled) return;
10
11     SecureScreenModule?.enableSecureMode();
12
13     return () => {
14       SecureScreenModule?.disableSecureMode();
15     };
16   }, [enabled]);
17 };
18
19 // Usage in sensitive component
20 const RecoveryPhraseScreen = () => {
21   useSecureScreen();
22
23   return <View>...</View>;
24 };
```

Third-party libraries

Consider using established libraries that handle cross-platform screenshot prevention:

- `react-native-screenshot-prevent` Provides FLAG_SECURE (Android) and blur overlay (iOS)
- `react-native-screen-capture-secure` Similar functionality with additional options

Implementation priority

Screen	Priority	Reason
Recovery phrase display	Critical	Direct mnemonic exposure
Recovery phrase input	Critical	Full mnemonic visible during entry
Transaction signing	High	Financial details visible
Receive addresses	Low	Addresses are semi-public

[M 302] Unsafe Onboarding Flows

	Level	Severity	Status
	3	Medium	Acknowledged

Category: User Experience & Education / Onboarding Safety

Affected Components: Seed phrase handling, clipboard copy, backup status prompts, user education/guidance

Description

The current onboarding emphasizes speed and convenience but does not ensure secure handling of the seed phrase or provide sufficient user education. Specific gaps include:

1. Inconsistent intent around seed phrase reveal and copying

- Observed: During seed phrase backup, the full seed phrase can be copied to the clipboard without revealing it, yet the flow cannot proceed unless the seed phrase is revealed on-screen. If the seed phrase is revealed then unrevealed (hidden), the user must reveal it again to proceed.
- Why problematic: Exposes the seed phrase unnecessarily, increasing the risk of shoulder surfing. The UX sends mixed signals about what is “required” for safety versus convenience.

2. Verification requires pasting/retyping the entire seed phrase

- Observed: The seed phrase confirmation step requires the user to paste or retype the full phrase to continue. The phrase is revealed by default, and there is no option to keep it hidden during verification.
- Why problematic: Asking for the full phrase encourages keeping the seed phrase in a digital/copyable form (clipboard, notes, password managers, keyboard caches/suggestions). Displaying the full seed phrase during verification further increases exposure time.

3. Clipboard copy lacks gating, warnings, and auto-clearance

- Observed:
 - A button copies the entire seed phrase to the system clipboard. There is no disclaimer, confirmation step, or any user guidance/education about clipboard risks.
 - After copying the seed phrase, it remains available in the system clipboard even after closing the app, and there is no “clear clipboard” action or timed auto-clear.
- Why problematic: Any malware or third-party apps (including keyboards/IMEs) with clipboard access could exfiltrate the seed phrase. The lack of gating or warnings increases the likelihood that users will copy the seed phrase without understanding the risks. Furthermore, persistent clipboard secrets increase the window for exfiltration or accidental disclosure.

4. No warning or protection against screenshots/screen capture on seed phrase screens

- Observed: There is no warning when screenshots / screen recordings are taken during seed phrase display or confirmation steps.
- Why problematic: Users may unintentionally store the seed phrase in photo galleries and cloud backups. This expands the attack surface beyond the wallet app itself, putting the seed phrase at risk even if the app is closed or uninstalled.

5. Missing seed phrase best-practice education before reveal/backup/verify

- Observed: No prominent education/warnings are shown before seed phrase reveal or during backup/verification (e.g., offline storage, avoiding digital copies, avoiding screenshots).
- Why problematic: Uninformed users may mishandle their seed phrase, increasing the risk of compromise or loss. Since onboarding is often the first interaction with the wallet, this is a critical moment to instill best practices. Omitting concise guidance increases the likelihood that users will choose unsafe storage and handling practices, putting their funds at risk.

6. No reminders about seed phrase backup after wallet setup

- Observed: After completing wallet setup, there are no reminders or alarms indicating that the wallet is not backed up. Note: there is a warning icon in the **Account Center** next to the unbacked wallet, and a more prominent warning after entering the wallet settings.
- Why problematic: Users may forget or neglect to back up their seed phrase, increasing the risk of losing access to their wallet if their device is lost or fails. Although there is a warning icon in the **Account Center** and a more prominent warning after entering the wallet settings, these may not be sufficient to prompt users to take action or may be overlooked by users who are not actively managing their wallet settings.

These issues do not necessarily constitute a direct remote exploit, but they materially increase the likelihood of seed-phrase compromise and irrecoverable loss through common real-world behaviors (clipboard use, screenshots, and “digital backup” shortcuts) during onboarding.

Recommendation

1. Align seed phrase gating with a consistent safety model

- Default seed phrase to hidden/blurred with a deliberate Tap to reveal action (already implemented).
- If the flow’s intent supports digital backup, allow users to complete verification without requiring an on-screen reveal.
- Otherwise, require reveal before copying, and allow proceeding without requiring the phrase to remain visible. Thus the flow becomes:

- reveal → copy (if desired) → re-hide (if desired) → proceed.

2. Replace full-paste confirmation with a randomized quiz

- Ask for 3 randomly selected words (positions vary per attempt).
- Re-randomize on failure and enforce a sensible attempt limit / short cooldown.
- Do not encourage storing the full seed digitally; avoid paste-from-clipboard UX here.

3. Implement explicit gating and warnings for clipboard copy

- Require an enabling action prior to copying (e.g., ticking a checkbox next to the `copy` button to enable copying) that triggers a modal risk explanation + confirmation (malware/clipboard access, best practices, preference for offline paper/metal backup).
- Implement automatic clipboard clearing after a short duration (e.g., 60 seconds) if the platform allows, and communicate this to users.
- If automatic clearing is not possible, provide clear guidance on manually clearing the clipboard.
- Keep a visible warning until the seed phrase is removed from the clipboard.

4. Add screenshot/screen-capture protections and/or strong warnings on seed phrase screens

- Display a clear warning: e.g., “Do not screenshot or screen-record your seed phrase.”
- Where platform-supported, prevent screenshots on seed phrase screens or obscure the seed phrase in screenshots/screen recordings (by blurring or a pop-up overlay displaying a warning message).
- Note: Refer to finding `301 - No Screenshot/Screen Recording Prevention` for implementation details.

5. Provide concise, staged seed phrase best-practice education

- Add brief, clear educational prompts at key stages:
 - In the onboarding intro: what a seed phrase is and why it matters, the importance of secure handling, and backup.
 - Before reveal: the criticality of seed phrase protection and privacy, offline storage, avoiding digital copies/screenshots, etc.
 - During backup/verify: best practices reminders
 - Before copy (or when enabling copy): clipboard risks and clearing guidance

6. Add persistent reminders about seed phrase backup after wallet setup

- Implement a more prominent and persistent reminder system to encourage users to back up their seed phrase after completing wallet setup. Specifically at these touch-points:
 - On the main dashboard/home screen (at the `portfolio` tab).
 - At the `Receive` tab, when trying to receive funds into the wallet.
 - At the `Send` tab, when trying to send funds from the wallet.

- At the `Settings` tab, when accessing general settings.
- Consider enforcing backup/verification when the user attempts to delete the wallet or perform critical actions.
- Implement periodic reminders or alarms to notify users that the wallet is not backed up (e.g., notifications, in-app prompts, or pop-ups).

[M 303] Missing Root and Jailbreak Detection

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Device Integrity / Resilience

Affected Files:

- `apps/lace-mobile/src/app/App.tsx`
- `apps/lace-mobile/src/app/load-app.ts`

Description

The application does not implement root detection for Android or jailbreak detection for iOS. On devices where operating system security restrictions have been modified or removed, the application sandbox protections may be bypassed, which can result in:

- Unauthorized access to application data through modified sandbox protections
- Reading Keychain (iOS) and Keystore (Android) entries
- Attaching runtime instrumentation tools (Frida, Cycript) to modify application behavior
- Direct filesystem access to Redux persisted state and MMKV data
- Bypassing SSL certificate pinning via interception tools
- Installation of system-level monitoring software

Security Model Changes on Modified Devices

On unmodified devices, applications operate within isolated sandboxes where the operating system enforces access controls. On rooted or jailbroken devices, these restrictions may be weakened or removed. For cryptocurrency wallet applications, this presents the following considerations:

1. Keychain/Keystore entries may be accessible to privileged processes
2. Runtime code can be modified via hooking frameworks
3. Memory contents may be dumped during sensitive operations
4. System-level software can monitor application activity

Example Attack Scenario

1. Application is installed on a device with modified system software
2. Privileged monitoring software operates in the background
3. During wallet creation or restoration:
 - Monitoring software intercepts SecureStore operations
 - Application memory is dumped during cryptographic operations
4. Sensitive data is extracted and transmitted externally
5. Funds may be transferred without user awareness

Relevant Risk Factors

- Mobile devices may be subject to physical theft
- Users may modify devices without full awareness of security implications

- Modified devices often install software from unofficial sources
- Development and testing workflows may utilize modified devices

References

- [OWASP MASTG-TEST-0045: Testing Root Detection \(Android\)](#)
- [OWASP MASTG-TEST-0088: Testing Jailbreak Detection \(iOS\)](#)
- [OWASP MASWE-0097: Root/Jailbreak Detection Not Implemented](#)
- [OWASP Mobile Application Security](#)

Recommendation

Implement device integrity detection with user notification rather than application blocking. This approach balances security awareness with user autonomy, while acknowledging that determined adversaries can bypass client-side detection.

Implementation Approach

1. Install a detection library:

```
1 npm install jail-monkey
```

bash

Alternative: `react-native-root-detection`

2. Perform device integrity checks on application startup (`apps/lace-mobile/src/app/App.tsx`):
 - Check for common modification indicators (Cydia, su binary, Magisk)
 - Detect runtime instrumentation frameworks (Frida, Xposed)
 - Calculate risk level based on detected indicators
3. Display warnings based on detected risk level:
 - High risk (jailbreak detected or hooks present): Display security warning dialog
 - Medium risk (multiple indicators): Show notification
 - Low risk (single indicator): Log for monitoring
4. Considerations:
 - Complete application blocking can be bypassed by motivated adversaries
 - Warning dialogs should use clear, non-technical language
 - Detection should not create a false sense of absolute security
 - Legitimate use cases (development, security research) should be considered

Example Implementation

```
1 // apps/lace-mobile/src/app/App.tsx
2 import JailMonkey from 'jail-monkey';
3
4 useEffect(() => {
```

typescript

```

5  const checks = {
6    isJailBroken: JailMonkey.isJailBroken(),
7    hookDetected: JailMonkey.hookDetected(),
8  };
9
10 let riskLevel = 'none';
11 if (checks.isJailBroken || checks.hookDetected) {
12   riskLevel = 'high';
13 }
14
15 if (riskLevel === 'high') {
16   Alert.alert(
17     'Security Notice',
18     'This device appears to have modified system software. Using a cryptocurrency
19     wallet on such devices may increase security risks, including potential
20     unauthorized access to wallet data.\n\nConsider using the application only on
21     devices with unmodified system software.',
22     [
23       { text: 'Exit Application', style: 'destructive' },
24       { text: 'Continue', style: 'default' }
25     ],
26     { cancelable: false }
27   );
28 }
29
30 // Log detection results
31 Sentry.captureMessage('Device integrity check', {
32   level: riskLevel === 'high' ? 'warning' : 'info',
33   extra: { checks, riskLevel }
34 });
35 }

```

Testing and Validation

After implementation, verify detection works correctly on:

- Unmodified devices (should show no warnings)
- Rooted Android devices with Magisk
- Jailbroken iOS devices with checkra1n or unc0ver
- Devices with Frida server running

Document the detection capabilities and limitations in user-facing security documentation.

[M 304] CRYPTO-01: PBKDF2 Insufficient Iterations (EMIP-3 Encryption)

	Level	Severity	Status
	3	Medium	Acknowledged

Description

Lace Mobile wallet uses PBKDF2-HMAC-SHA512 with 19,162 iterations to derive encryption keys from user passwords (via EMIP-3 specification). This is approximately 31x below OWASP 2023 recommended standards (600,000 iterations minimum), which may expose user passwords to offline brute-force attacks if encrypted wallet data is exfiltrated.

Low iteration count enables password brute-forcing

- Observed: `@cardano-sdk/key-management` EMIP-3 implementation uses only 19,162 iterations: <https://github.com/input-output-hk/cardano-js-sdk/blob/d705b98d2af32da851792441a78a0a0b681cf61f/packages/key-management/src/emip3.ts#L8>
- Usage in lace-mobile:
 1. Mobile app loads `vaultInMemory` module:
 - `apps/lace-mobile/src/app/util/all-modules.ts:27,61`
 2. User creates/restores wallet via:
 - `packages/module/vault-in-memory/src/onboarding-options-mobile.tsx`
 3. Calls `createInMemoryWalletEntityFactory` in:
 - `packages/module/vault-in-memory/src/util.ts:37-42`
 - encrypts recovery phrase with `emip3encrypt`
 4. For Cardano:
 - `packages/module/blockchain-cardano/src/in-memory-wallet-integration.ts:35-49` calls `InMemoryKeyAgent.fromBip39MnemonicWords` which internally uses `emip3encrypt` for `encryptedRootPrivateKeyBytes`
 5. For Bitcoin: `packages/module/blockchain-bitcoin/src/in-memory-wallet-integration.ts:41` calls `emip3encrypt(seed, password)` directly to encrypt the Bitcoin seed
- Why problematic: PBKDF2 derives the encryption key from the user password using only 19,162 iterations. An attacker who obtains encrypted wallet data (via malware, backup theft, device theft, or storage breach) can test millions of password candidates per second using GPU/ASIC clusters. With approximately 31x fewer iterations than OWASP 2023 recommended standards, weak passwords may be cracked in hours instead of years. Once the password is recovered, the attacker calls `emip3decrypt` to recover all private keys and seeds.

Attack flow

1. Attacker obtains encrypted keys/seed from device storage (malware, device theft, backup extraction)
 - See related finding `401 - Encrypted Seed Stored in Unprotected File Storage`
2. Attacker brute-forces password: `PBKDF2(candidate, salt, 19162)` - test decryption
3. Once password found: `emip3decrypt(encryptedData, password)` - recover private keys/seeds

4. Attacker drains wallet funds across all supported blockchains (Cardano, Bitcoin)

Code evidence

Recovery phrase encryption:

- `packages/module/vault-in-memory/src/util.ts`:

```
1 // Line 37-42: Encrypt recovery phrase with password typescript
2 const encryptRecoveryPhrase = async (
3   recoveryPhrase: string[],
4   password: Uint8Array,
5 ) => {
6   const walletEncrypted = await emip3encrypt(
7     ByteArray.fromUTF8(util.joinMnemonicWords(recoveryPhrase)),
8     password,
9   );
10  return HexBytes.fromByteArray(ByteArray(walletEncrypted));
11 };
```

Bitcoin seed encryption:

- `packages/module/blockchain-bitcoin/src/in-memory-wallet-integration.ts`

```
1 // Line 40-42: Encrypt Bitcoin seed with password typescript
2 const encryptBitcoinSeed = async (seed: Buffer, passphrase: Uint8Array) => {
3   const seedEncrypted = await emip3encrypt(seed, passphrase); // Password protects seed
4   return HexBytes.fromByteArray(seedEncrypted);
5 };
```

EMIP-3 implementation (cardano-js-sdk):

```
1 // @cardano-sdk/key-management - emip3.ts typescript
2 const PBKDF2_ITERATIONS = 19_162; // ~31x below OWASP 2023 recommended minimum
3
4 const derivedKey = await crypto.subtle.deriveBits(
5   {
6     name: 'PBKDF2',
7     hash: 'SHA-512',
8     salt,
9     iterations: PBKDF2_ITERATIONS // Only 19,162 iterations
10  },
11  passwordKey,
12  32 * 8
13 );
```

Mobile-specific risk factors

- Device theft: Mobile devices are more susceptible to physical theft than desktop/browser environments
- Backup extraction: iOS/Android backup mechanisms may expose encrypted wallet data if device backups are compromised
- Combined with finding 401: If the unencrypted MMKV (02) finding is exploited, the attacker can extract encrypted wallet data and then brute-force the weak PBKDF2-derived key to fully compromise all wallet secrets
- Password prompt has no attempt throttling (conditional): When the password flow is used, failed attempts immediately return to the password prompt without rate limiting (`packages/contract/authentication-prompt/src/store/state-machine.ts`). On mobile, the password flow is typically only reachable when biometrics are unavailable and `ENFORCE_BIOMETRIC_REQUIREMENT` is disabled (`packages/contract/authentication-prompt/src/store/side-effects/side-effects.ts` , `apps/lace-mobile/src/app/feature-flags.ts`), so this is a defense-in-depth gap rather than a primary risk.

Recommendation

1. Increase PBKDF2 iterations to meet current security standards
 - Update `PBKDF2_ITERATIONS` to 600,000 iterations (OWASP 2023 recommended minimum for PBKDF2-HMAC-SHA512) in `cardano-js-sdk` EMIP-3 implementation.
 - Benchmark performance impact on target mobile devices (iOS/Android) to ensure acceptable user experience.
 - Coordinate with Cardano SDK maintainers for ecosystem-wide implementation.
2. Consider migration to Argon2id (memory-hard KDF)
 - Argon2id provides resistance against GPU/ASIC attacks through memory-hard properties (recommended: 64MB memory, 3 iterations, parallelism=4).
 - Evaluate compatibility with existing EMIP-3 encrypted data and migration path.
 - Note: May have performance implications on lower-end mobile devices.
3. Implement password strength requirements
 - As a defense-in-depth measure, enforce minimum password complexity to increase brute-force difficulty even with suboptimal iteration counts.
4. Add attempt throttling/lockout for password verification (defense-in-depth)
 - If the password flow is enabled (e.g., biometrics unavailable and enforcement disabled), implement exponential backoff or temporary lockout after repeated failures.

Related Findings

- [H 401] Encrypted Seed Stored in Unprotected File Storage

Finding 401 allows extraction of encrypted wallet data from device storage. CRYPTO-01 then enables brute-forcing the weak PBKDF2-derived key to decrypt the extracted data.

[M 305] DEP-02: Vulnerable Dev Dependencies

Level	Severity	Status
3	Medium	Acknowledged

Category: Dependencies / Supply Chain / Build Chain

Description

Dev dependencies do not typically ship inside the final APK/IPA, but they do matter for supply-chain security because they are installed and executed on developer machines and CI runners (tests, bundlers, CLIs, codegen), and can therefore affect build integrity.

This finding captures only the vulnerabilities that appear when dev dependencies are included.

Commands used:

- Full (includes dev deps): `npm audit --workspace lace-mobile --json`
- Runtime-focused (omits dev deps): `npm audit --workspace lace-mobile --omit=dev --json`

Dev-only delta (present in full audit, absent with `--omit=dev`): 8 advisories (1 critical, 2 high, 1 moderate, 4 low).

Effective impact on lace-mobile: After triage, lace-mobile is only affected by `vite` (moderate) via `vitest` and low-severity dev server issues. The critical `happy-dom` RCE is not used by lace-mobile tests. The high-severity `ecpair` / `valibot` issues are in the blockchain-bitcoin module, not lace-mobile.

Triage Results

The table below categorizes each advisory by impact context and relevance to lace-mobile:

Package	Severity	Affects Lace-Mobile	Installed Version	Vulnerable Range	Used By	Notes
happy-dom	Critical	No	18.0.1	<20.0.0	Root devDependencies	VM Context Escape → RCE. lace-mobile uses environment: 'node', not happy-dom.
valibot	High	No	0.37.0	≥0.31.0 <1.2.0	ecpair (Bitcoin testing)	ReDoS in emoji regex. Only in @lace-module/blockchain-bitcoin, not lace-mobile.
ecpair	High	No	3.0.0	≥3.0.0-rc.0	blockchain-bitcoin devDeps	Affected via valibot. Only in blockchain-bitcoin module.
vite	Moderate	Yes (dev)	6.3.3	6.0.0 - 6.4.0	vitest (test runner)	FS access bypass. Affects devs running vitest locally.
cookie	Low	Yes (dev)	0.6.0	<0.7.0	@remix-run/server-runtime	Out-of-bounds chars. Low severity, expo-atlas dev server only.
@remix-run/server-runtime	Low	Yes (dev)	2.16.0	≤2.16.3	@remix-run/node	Affected via cookie. expo-atlas dev server.
@remix-run/node	Low	Yes (dev)	2.16.0	≤2.16.3	@expo/server → expo-atlas	lace-mobile has expo-atlas in devDeps for bundle analysis.
morgan	Low	Yes (dev)	1.10.0	1.6.0 - 1.10.0	expo-atlas, Appium	HTTP logging via vulnerable on-headers. Dev servers only.

Summary by Impact Category (Lace-Mobile Focus)

Not Applicable to Lace-Mobile

1. `happy-dom` (Critical upstream) - VM Context Escape → RCE
 - Location: Root `package.json` devDependencies

- Vulnerability: VM sandbox escape allows arbitrary code execution
- Lace-Mobile Status: NOT AFFECTED - lace-mobile's `vitest.config.js` inherits from `vitest.base.config.js` which uses `environment: 'node'`, not `happy-dom`
- Monorepo Note: Other workspace packages using `happy-dom` for DOM testing are at risk; upgrade recommended at root level

2. `valibot` + `ecpair` (High upstream) - ReDoS

- Location: `packages/module/blockchain-bitcoin/package.json` `devDependencies`
- Vulnerability: ReDoS in emoji validation regex
- Lace-Mobile Status: NOT AFFECTED - this is in the blockchain-bitcoin module, not lace-mobile's dependency graph

Affects Lace-Mobile Development (Moderate)

3. `vite` (Moderate) - Filesystem Access Bypass

- Location: Transitive via `vitest`
- Vulnerability: Dev server allows access to files outside project root
- Impact: Malicious test files or local network attacks could read sensitive files
- Lace-Mobile Status: AFFECTS DEVS - impacts developers running `vitest` locally
- Recommendation: Upgrade `vitest` when it updates `vite` dependency

Affects Lace-Mobile Development (Low)

4. `cookie`, `@remix-run/*`, `morgan` (Low)

- Location: Transitive via `expo-atlas` (lace-mobile devDep)
- Vulnerability: Various low-severity issues in HTTP server tooling
- Impact: Dev server quirks, header manipulation
- Lace-Mobile Status: Only affects developers using `expo-atlas` for bundle analysis
- Recommendation: Monitor for upstream fixes

Lace-Mobile Specific Analysis

What dev tools does lace-mobile use?

Tool	Declared In	Purpose	Affected by Vulnerabilities
<code>vitest</code>	Root devDeps, used in <code>project.json</code>	Unit testing	Yes, via <code>vite</code> (moderate)
<code>expo-atlas</code>	<code>apps/lace-mobile/devDependencies</code>	Bundle size analysis	Yes, via <code>@remix-run/*</code> , <code>morgan</code> (low)
<code>babel-plugin-module-resolver</code>	<code>apps/lace-mobile/devDependencies</code>	Build config	No
<code>@lace-lib/util-dev-app</code>	<code>apps/lace-mobile/devDependencies</code>	Dev utilities	No known issues

What does NOT affect lace-mobile?

1. `happy-dom`: lace-mobile tests use `environment: 'node'`, not a DOM environment
2. `ecpair` / `valibot`: Only in `blockchain-bitcoin` module devDeps, not lace-mobile
3. Appium packages: Only used by E2E test workspaces (`lace-mobile-e2e`), not lace-mobile itself

Recommendations for Lace-Mobile

Lace-Mobile Specific Actions

1. Upgrade `vitest` when available (Moderate priority)
 - Monitor for `vitest` release that includes `vite@6.4.1+`
 - Or pin `vite` resolution in root `package.json`
 - This is the only moderate-severity issue affecting lace-mobile developers
2. Expo Atlas updates (Low priority)
 - Wait for `expo-atlas` to update its dependencies
 - `@remix-run/*` and `morgan` vulnerabilities are low severity
 - Only affects bundle analysis tooling

Monorepo-Wide Actions (Out of Lace-Mobile Scope)

3. Upgrade `happy-dom` to $\geq 20.0.0$
 - Not required for lace-mobile - it doesn't use happy-dom
 - Recommended for monorepo health: the RCE vulnerability affects other workspace tests using DOM environment
 - Root `package.json`: `"happy-dom": "^20.0.0"`
4. Track `ecpair` / `valibot` fix
 - Not required for lace-mobile - this is in blockchain-bitcoin module
 - Requires semver-major upgrade of `ecpair` (v3 → when patched)

CI Hardening (Already in Place)

5. Maintain `--ignore-scripts` in CI
 - Already in place: `ci/setup.sh` runs `npm ci --ignore-scripts`
 - This reduces risk from lifecycle script attacks
6. Consider isolated test environments (Optional)
 - For tests processing untrusted inputs, use containerized runners
 - Limits blast radius of any VM escape vulnerabilities

Notes / Why This Matters

- CI builds install dev deps by default unless `npm ci --omit=dev` is used. In this repo, `ci/setup.sh` runs `npm ci --ignore-scripts`, which reduces risk from lifecycle scripts but does not eliminate risk when vulnerable tools are executed (tests/dev servers).
- Developer workstation exposure: vulnerable dev servers and test tooling can be exposed to hostile inputs (local network, malicious test fixtures, untrusted PRs).
- Artifact integrity: compromise of CI/dev tooling can lead to tampered release artifacts even when the vulnerable package is not bundled into the app.
- Monorepo scope: Some vulnerabilities (like `happy-dom`) are in root devDependencies but may not affect specific workspaces like lace-mobile that don't use them.

[M 306] NUMERIC-01: Bitcoin Satoshi Calculations Use JavaScript Number Type

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Numeric Safety

Affected Files:

- packages/contract/bitcoin-context/src/bitcoin-data-provider.ts
- packages/module/blockchain-bitcoin/src/wallet/BitcoinWallet.ts
- packages/module/blockchain-bitcoin/src/input-selection/GreedyInputSelector.ts
- packages/module/blockchain-bitcoin/src/store/side-effects.ts

Description

The Bitcoin blockchain module uses JavaScript's `number` type for satoshi (smallest Bitcoin unit) calculations throughout the codebase. While JavaScript's `Number.MAX_SAFE_INTEGER` (9,007,199,254,740,991) exceeds Bitcoin's maximum supply in satoshis (2.1 quadrillion), using `number` instead of `bigint` for financial calculations is a risky design pattern that can lead to precision loss in aggregate operations.

Affected Type Definitions

```
1 // `packages/contract/bitcoin-context/src/bitcoin-data-provider.ts` typescript
2 // Line 90
3 export type BitcoinUTx0 = {
4   readonly satoshis: number; // Should be bigint
5   // ...
6 };
7
8 // Line 105
9 export type BitcoinInputEntry = {
10   readonly satoshis: number; // Should be bigint
11   // ...
12 };
13
14 // Line 122
15 export type BitcoinOutputEntry = {
16   readonly satoshis: number; // Should be bigint
17   // ...
18 };
```

Affected Calculations

1. Wallet balance calculation uses number accumulator:

```
1 // `packages/module/blockchain-bitcoin/src/wallet/BitcoinWallet.ts` (line 159)
2 this.utxos$
3   .pipe(
4     map(utxos => utxos.reduce((total, utxo) => total + utxo.satoshis, 0)),
5     // Accumulator starts as number (0), sum is number
6   )
7   .subscribe(balance => {
8     this.balance$.next(balance);
9   });
```

2. Input selection uses number for all satoshi math:

```
1 // `packages/module/blockchain-bitcoin/src/input-selection/GreedyInputSelector.ts`
2 // Lines 41-46
3 const totalOutput = outputs.reduce(
4   (accumulator, output) => accumulator + output.value,
5   0, // number accumulator
6 );
7 let inputSum = 0; // number
8 let fee = 0; // number
9
10 // Line 50 - satoshi accumulation
11 inputSum += utxo.satoshis;
12
13 // Line 63 - change calculation
14 let change = inputSum - totalOutput - fee;
```

3. Activity mapping sums satoshis as numbers:

```
1 // `packages/module/blockchain-bitcoin/src/store/side-effects.ts` (lines 53-73)
2 let totalInputFromUser = 0; // number
3 let totalOutputToUser = 0; // number
4
5 for (const input of rawTx.inputs) {
6   if (userAddresses.has(input.address)) {
7     totalInputFromUser += input.satoshis; // number addition
8   }
9 }
10
```

```
11 // Only converts to BigInt AFTER all number math is done
12 const netAmount = BigInt(totalOutputToUser - totalInputFromUser);
```

Risk Analysis

Number.MAX_SAFE_INTEGER = 9,007,199,254,740,991

- Bitcoin max supply: 21,000,000 BTC = 2,100,000,000,000,000 satoshis
- MAX_SAFE_INTEGER in BTC: 90,071,992 BTC

While a single balance won't exceed MAX_SAFE_INTEGER, risks exist in:

1. Aggregate calculations: Summing many large UTXOs or historical transactions
2. Intermediate values: Calculations that multiply or add many values before final result
3. Future-proofing: Code may be reused for other tokens with different decimal places
4. Precision edge cases: Values near MAX_SAFE_INTEGER lose precision silently

Example of silent precision loss:

```
1 const a = 9007199254740992; // MAX_SAFE_INTEGER + 1
2 const b = 9007199254740993; // MAX_SAFE_INTEGER + 2
3 a === b // true! Silent precision loss
```

Recommendation

1. Change satoshi type definitions to `bigint`:

```
1 export type BitcoinUTx0 = {
2   readonly satoshis: bigint;
3   // ...
4 };
```

2. Update all calculations to use `BigInt`:

```
1 // Balance calculation
2 map(utxos => utxos.reduce((total, utxo) => total + utxo.satoshis, 0n))
3
4 // Input selection
5 const totalOutput = outputs.reduce(
6   (acc, output) => acc + output.value,
7   0n,
8 );
9 let inputSum = 0n;
10 let fee = 0n;
```

3. Update activity mapping:

```
1  let totalInputFromUser = 0n;
2  let totalOutputToUser = 0n;
3
4  for (const input of rawTx.inputs) {
5    if (userAddresses.has(input.address)) {
6      totalInputFromUser += input.satoshis;
7    }
8  }
9
10 const netAmount = totalOutputToUser - totalInputFromUser;
```

typescript

4. Update provider implementations to return satoshis as `bigint` (or convert at the boundary)

Related Findings

- NUMERIC-02: Number/BigInt Mixing in Bitcoin Input Selection
- NUMERIC-03: rawToBigInt parseInt Precision Loss

[M 307] NUMERIC-02: Number/BigInt Mixing in Bitcoin Input Selection

Level	Severity	Status
3	Medium	Acknowledged

Category: Numeric Safety

Description

The `GreedyInputSelector` class contains code that compares JavaScript `number` values with `BigInt` values using comparison operators. While modern JavaScript (ES2020+) allows this comparison through implicit type coercion, it represents inconsistent type handling that could lead to unexpected behavior in edge cases and indicates a design flaw in the numerical handling approach.

`packages/module/blockchain-bitcoin/src/input-selection/GreedyInputSelector.ts`

Mixed Type Comparison

```

1 // Lines 135-140
2 const newChange = newInputSum - totalOutput - newFee; // newChange is number
3
4 const rescued = newChange - originalChange; // rescued is number
5 const extraCost = BigInt(Math.ceil(INPUT_SIZE * feeRate)); // extraCost is BigInt
6
7 if (rescued >= extraCost && newChange >= BigInt(DUST_THRESHOLD)) {
8     // ^^^^^^          ^^^^^^^
9     // number >= BigInt    number >= BigInt
10    // ...
11 }
```

Type Analysis

Variable	Type	Source
<code>newInputSum</code>	<code>number</code>	<code>inputSum + utxo.satoshis</code> (line 127)
<code>totalOutput</code>	<code>number</code>	<code>outputs.reduce(..., 0)</code> (lines 41-44)
<code>newFee</code>	<code>number</code>	<code>this.computeFee(...)</code> returns <code>number</code> (line 104)
<code>newChange</code>	<code>number</code>	arithmetic of numbers
<code>rescued</code>	<code>number</code>	<code>newChange - originalChange</code>
<code>extraCost</code>	<code>BigInt</code>	<code>BigInt(Math.ceil(...))</code> (line 138)
<code>BigInt(DUST_THRESHOLD)</code>	<code>BigInt</code>	explicit conversion

JavaScript Behavior

In ES2020+, mixed comparisons between `number` and `BigInt` are allowed:

```
1 5 >= BigInt(3) // true (5 is coerced for comparison)
2 5n >= 3 // true (3 is coerced for comparison)
```

However, this has edge-case issues:

```
1 Number.MAX_SAFE_INTEGER >= BigInt(Number.MAX_SAFE_INTEGER) // true
2 Number.MAX_SAFE_INTEGER + 1 >= BigInt(Number.MAX_SAFE_INTEGER + 2) // true! (wrong)
3 // Because Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2 in JS
```

Risk Assessment

1. Inconsistent type handling: Code mixes types arbitrarily
2. Edge case bugs: Comparisons near MAX_SAFE_INTEGER could fail silently
3. Maintenance hazard: Developers may assume consistent typing
4. Arithmetic incompatibility: You cannot do `number + BigInt` (throws `TypeError`), but you can compare them, leading to confusion

Why This Code Works (For Now)

The current values involved are small enough that precision loss is unlikely:

- `INPUT_SIZE = 68`
- `DUST_THRESHOLD = 546`
- `feeRate` is typically small (1-100 sat/vB)

But this is fragile code that relies on small values to avoid issues.

Recommendation

1. Unify on `BigInt` throughout the input selection logic:

```
1 export class GreedyInputSelector implements InputSelector {
2   public selectInputs(props: InputSelectorProps): InputSelectionResult | null {
3     const { changeAddress, utxos, outputs, feeRate, hasOpReturn } = props;
4
5     const selected: BitcoinUTx0[] = [];
6     const totalOutput = outputs.reduce(
7       (accumulator, output) => accumulator + BigInt(output.value),
8       0n, // BigInt accumulator
9     );
10    let inputSum = 0n; // BigInt
11    let fee = 0n; // BigInt
12
13    for (const utxo of utxos) {
14      selected.push(utxo);
15      inputSum += BigInt(utxo.satoshis);
```

```

16     fee = this.computeFee({
17         inputCount: selected.length,
18         outputCount: outputs.length + (hasOpReturn ? 1 : 0),
19         feeRate,
20         hasOpReturn,
21     });
22
23     if (inputSum >= totalOutput + fee) break;
24 }
25 // ...
26 }
27
28 private computeFee(params: ComputeFeeParams): bigint {
29     const { inputCount, outputCount, feeRate, hasOpReturn } = params;
30     const size = BigInt(
31         inputCount * INPUT_SIZE +
32         (outputCount + (hasOpReturn ? 1 : 0)) * OUTPUT_SIZE +
33         TRANSACTION_OVERHEAD
34     );
35     // Ceiling division for BigInt
36     const feeRateBigInt = BigInt(Math.ceil(feeRate));
37     return (size * feeRateBigInt);
38 }
39 }

```

2. Update the dust rescue logic to use consistent types:

```

1 private attemptDustRescue({                                typescript
2     change,
3     feeRate,
4     // ...
5 }: RescueDustParams): { change: bigint; fee: bigint } {
6     if (change === 0n || change >= BigInt(DUST_THRESHOLD)) {
7         return { change, fee: 0n };
8     }
9
10    // ... all calculations using BigInt
11 }

```

3. Consider TypeScript strict mode additions:

- Enable `noImplicitAny`

- Use branded types for satoshis:

```
type Satoshis = bigint & { readonly _brand: unique symbol };
```

Related Findings

- NUMERIC-01: Bitcoin satoshi calculations use JavaScript number type
- NUMERIC-03: rawToBigInt parseInt Precision Loss

[M 308] BUILD-03: iOS Build Uses Remote Podspec Without Integrity Verification

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Build Configuration / Supply Chain

Affected Files:

- `apps/lace-mobile/expo-build-plugin/src/pod-dependencies.json`
- `apps/lace-mobile/expo-build-plugin/src/ios.ts`

Description

The iOS build configuration references `ApolloLibrary` via a remote podspec URL:

```
1 https://raw.githubusercontent.com/hyperledger-identus/apollo/644bb6d286f6298310f7eaea3ef84be40c39f605/ApolloLibrary.podspec
```

While the URL is pinned to a specific commit hash (`644bb6d`), CocoaPods does not verify cryptographic integrity of the fetched content. In addition, the podspec itself downloads a binary release artifact via `:http` without a checksum, e.g.:

```
1 spec.source = {
2   :http => 'https://github.com/hyperledger-identus/apollo/releases/download/v1.7.2/ApolloLibrary.xcframework.zip'
3 }
```

This means the overall integrity depends on:

1. GitHub's infrastructure integrity
2. The commit hash remaining valid and unchanged (for the podspec)
3. Network security during fetch (HTTPS)

Because the binary is fetched from a GitHub Release URL and no checksum is enforced, a compromised release asset (or a replaced asset) could silently change the code that ships in the app.

Recommendation

1. Prefer vendoring or building Apollo from source pinned to a specific commit (rather than downloading a binary over HTTP).
2. If continuing to use a remote binary:
 - Require a checksum in the podspec `spec.source` (e.g. `:sha256` / `:sha1`) and pin it to a known-good value.
 - Add a CI pre-build verification step to download the `.xcframework.zip`, compute SHA-256, and compare against a committed known-good hash.
3. Document and gate updates:

- Any change to the Apollo iOS dependency must include an updated hash and reviewer sign-off.
- Keep `Podfile.lock` (or an equivalent lock artifact) stable across CI runs and alert on unexpected diffs.

[M 309] BUILD-04: Sentry Initialized with Debug Mode and Full Sampling

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Build Configuration / Observability

Affected File: `apps/lace-mobile/src/app/App.tsx`

Description

The Sentry observability SDK is initialized with debug settings that are inappropriate for production:

```
1 initializeObservability(ENV, {
2   dsn: process.env.EXPO_PUBLIC_SENTRY_DSN,
3   debug: true, // Debug logging enabled
4   tracesSampleRate: 1.0, // 100% trace sampling
5   profilesSampleRate: 1.0, // 100% profile sampling
6   sendDefaultPii: false,
7 });
```

typescript

Issues:

- `debug: true` enables verbose console logging which may leak information
- 100% sampling rates capture all transactions and profiles, increasing:
 - Data exposure surface (more events = more potential secret leakage in breadcrumbs)
 - Performance overhead
 - Telemetry costs

While `sendDefaultPii: false` is correctly set, any sensitive data that reaches Sentry breadcrumbs, tags, or context will be transmitted.

Recommendation

1. Set `debug: false` for production builds (e.g. `ENV === 'production' ? false : true`, or simply `debug: __DEV__`)
2. Reduce sampling rates for production:
 - `tracesSampleRate: 0.1` (10%) or lower
 - `profilesSampleRate: 0.1` (10%) or lower
3. Implement environment-specific configuration:

```
1 const isProduction = ENV === 'production';
2 initializeObservability(ENV, {
3   debug: !isProduction,
4   tracesSampleRate: isProduction ? 0.1 : 1.0,
5   profilesSampleRate: isProduction ? 0.1 : 1.0,
```

typescript

```
6 });
```

4. Audit Sentry's `beforeSend` and `beforeBreadcrumb` hooks to scrub any sensitive data
5. Review what data is attached to Sentry events (user context, tags, breadcrumbs)

[L 201] No SSL/Certificate Pinning

	Level	Severity	Status
	2	Low	Acknowledged

Affected Components: Network layer across providers

Description

The Lace Mobile application does not implement SSL/TLS certificate pinning for network communications with sensitive backend services. This allows man-in-the-middle (MITM) attacks when the device's trust store is compromised through user-installed CA certificates, corporate proxy configurations, or device compromise. The impact is primarily confidentiality/integrity of API responses (balances, UTXOs, fee estimates, tx status, analytics).

Affected network endpoints

The application communicates with several sensitive services without certificate pinning:

1. Blockchain providers
 - Blockfrost API - Transaction submission, UTXO queries, address history
 - Maestro API - Alternative blockchain data provider
2. Observability services
 - Sentry - Error reporting and crash logs
 - PostHog - Analytics and usage telemetry
3. Other services
 - Any GraphQL/REST APIs used for wallet functionality

Attack scenario

1. Corporate/institutional MITM
 - User's organization installs corporate CA certificate on device
 - Corporate proxy can decrypt and inspect all app traffic
 - Transaction data, addresses, and usage patterns are visible
2. Malicious Wi-Fi attack
 - Attacker sets up rogue Wi-Fi access point
 - Tricks user into installing "network certificate" for connectivity
 - All app traffic can be intercepted and modified
3. Compromised device
 - Malware with root access installs CA certificate in system trust store
 - App traffic is decrypted by malware
 - Transaction details, addresses, and balance checks are exposed
4. Traffic manipulation
 - MITM attacker modifies API responses
 - Could manipulate displayed balances or transaction status

- Could inject malicious receive addresses (though this requires additional UI manipulation)

Verification

Install a user CA certificate and proxy traffic using a tool or script:

```
1 # 1. Export proxy CA certificate
2 # 2. Install on test device (Settings > Security > Install certificate)
3 # 3. Configure device to use proxy
4 # 4. Launch Lace Mobile and observe traffic
```

If certificate pinning is absent, all HTTPS traffic will be visible in the proxy tool.

Platform caveats:

- Android (targetSdk 35): User-installed CAs are not trusted by default for release builds. MITM typically requires a debug build (or a network security config that opts into user CAs) or a rooted device.
- iOS: User-installed CAs only work after explicitly enabling full trust for the certificate in Settings. Without that, the proxy will fail.

Current state

No implementation of certificate pinning was found in the codebase. A search for pinning-related implementations (TrustManagerFactory, certificate pinning, SSL pinning) returned no relevant results.

Recommendation

Option 1: React Native TrustKit (iOS) + OkHttp Pinning (Android)

Android - OkHttp Certificate Pinning:

```
1 // NetworkModule.kt or equivalent
2 val certificatePinner = CertificatePinner.Builder()
3   .add("blockfrost.io", "sha256/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=")
4   .add("maestro.api", "sha256BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB=")
5   .add("sentry.io", "sha256/CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC=")
6   .build()
7
8 val client = OkHttpClient.Builder()
9   .certificatePinner(certificatePinner)
10  .build()
```

iOS - TrustKit Configuration:

```
1 // AppDelegate.swift
```

```

2 let trustKitConfig: [String: Any] = [
3   kTSKSwizzleNetworkDelegates: true,
4   kTSKPinnedDomains: [
5     "blockfrost.io": [
6       kTSKEnforcePinning: true,
7       kTSKPublicKeyHashes: [
8         "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=",
9         "backup-pin-hash-here="
10      ]
11    ],
12    "maestro.api": [
13      kTSKEnforcePinning: true,
14      kTSKPublicKeyHashes: [
15        "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB="
16      ]
17    ]
18  ]
19 ]
20 TrustKit.initSharedInstance(withConfiguration: trustKitConfig)

```

Option 2: React Native SSL Pinning Library

Use `react-native-ssl-pinning` or similar:

```

1 import { fetch } from 'react-native-ssl-pinning';
2
3 const response = await fetch(url, {
4   method: 'GET',
5   sslPinning: {
6     certs: ['blockfrost', 'maestro'] // Certificate files in assets
7   }
8 });

```

Option 3: Network Security Config (Android) + ATS (iOS)

Android - `network_security_config.xml`:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <network-security-config>
3   <domain-config>
4     <domain includeSubdomains="true">blockfrost.io</domain>
5     <pin-set>

```

```

6      <pin digest="SHA-256">AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA= </pin>
7      <pin digest="SHA-256">backup-pin-here= </pin>
8      </pin-set>
9      </domain-config>
10 </network-security-config>

```

iOS - Info.plist (partial protection via ATS):

iOS App Transport Security provides baseline protection but not full certificate pinning. Combine with TrustKit for complete coverage.

Implementation considerations

1. Pin to intermediate certificates, not leaf certificates
 - Leaf certificates rotate frequently
 - Intermediate CA certificates are more stable
 - Reduces app update frequency for pin rotation
2. Include backup pins
 - Always include at least one backup pin
 - Prevents app lockout if primary certificate is rotated
 - Document pin rotation procedure
3. Implement pin failure reporting
 - Log pinning failures for security monitoring
 - Can help detect active MITM attacks
 - Don't leak sensitive details in error messages
4. Graceful degradation strategy
 - Define behavior when pinning fails
 - Consider: block request, warn user, or allow with logging
 - For wallet apps, blocking is recommended
5. Testing in development
 - Disable pinning in debug builds or use debug certificates
 - Ensure CI/CD can still run tests with mocked network
 - Document how to test with real endpoints

Priority by endpoint

Endpoint	Priority	Reason
Blockchain APIs	High	Transaction data, balance info
Transaction submission	Critical	Financial operations
Sentry/PostHog	Medium	May contain sensitive metadata
Other APIs	Low-Medium	Depends on data sensitivity

Related Findings

- BUILD-05 (Low-Medium): Bundled API keys - exposed API keys combined with lack of pinning could allow impersonation
- BUILD-04 (Medium): Sentry debug mode - if Sentry traffic is intercepted, debug-level data may be exposed

[L 202] Wallet Renaming Without Re-authentication

	Level	Severity	Status
	2	Low	Acknowledged

Category: Settings Management / Authentication

Affected Components: Wallet Settings (Renaming)

Description

Wallet name can be changed without re-authentication

- Observed: Wallet name can be modified without re-authentication.
- Why problematic: Allows changes to a wallet's identity label without re-authentication, making wallet labels less reliable and increasing the risk of confusion or user error if the device/session is briefly accessed locally.
- Example attack scenario: On a shared or workplace device where multiple wallets are used for different purposes, an attacker with momentary access renames wallets to misleading labels. The legitimate user later relies on the names when sending funds and inadvertently uses the wrong wallet, resulting in improper fund allocation or loss.

Recommendation

Require re-authentication for wallet identity-affecting settings

- Require re-authentication before changing the wallet name and other identity-affecting settings to prevent unauthorized local tampering.
- Provide a clear display message indicating the wallet name has been changed, to alert users to potential unauthorized changes.

[L 203] DEP-01: Vulnerable NPM Dependencies

Level	Severity	Status
2	Low	Acknowledged

Affected Components: lace-mobile workspace dependency graph

Description

`npm audit` reports vulnerable dependencies in the `lace-mobile` workspace dependency graph.

Command used (non-dev scope): `npm audit --workspace lace-mobile --omit=dev --json`

Summary (non-dev scope): 14 total advisories (2 critical, 4 high, 2 moderate, 6 low).

Effective impact on mobile runtime: After triage, only 1 low-severity issue (`min-document`) affects the mobile app runtime. The critical `sha.js` vulnerability is mitigated by Metro's redirect to native crypto. All other issues are build-time only or development-only.

Triage Results

The table below categorizes each advisory by where the vulnerability impacts the lace-mobile app:

Package	Severity	Installed Version	Vulnerable Range	Affects Runtime	Source Chain	Notes
<code>sha.js</code>	Critical	2.4.11	≤2.4.11	No (mitigated)	<code>@cardano-sdk/key-management</code> → <code>pbkdf2</code> → <code>sha.js</code>	Not applicable to mobile. Metro redirects <code>pbkdf2</code> to <code>react-native-quick-crypto</code> on native, so iOS/Android builds use native crypto and never execute this vulnerable code path. Only affects web builds (out of scope for lace-mobile).
<code>form-data</code>	Critical	3.0.3	≥3.0.0 <3.0.4	No	<code>@expo/cli</code> → <code>form-data</code>	Used by Expo CLI during development/build. Not shipped in app bundle.
<code>glob</code>	High	10.4.5	≥10.2.0 <10.5.0	No	<code>@expo/cli</code> , <code>@expo/config-plugins</code> , <code>@expo/metro-config</code> → <code>glob</code>	CLI command injection in glob binary. Only used during bundling. Not in runtime.
<code>image-size</code>	High	1.2.0	≥1.1.0 <1.2.1	No	<code>metro</code> → <code>image-size</code>	Metro uses this to determine asset dimensions at bundle time. Not shipped in app.
<code>jws</code>	High	3.2.2	<3.2.3	Conditional	<code>@redux-devtools/remote</code> → <code>socketcluster-client</code> → <code>ag-auth</code> → <code>jsonwebtoken</code> → <code>jws</code>	Only loads when <code>ENV === 'development'</code> . Production builds should not include this code path.
<code>node-forge</code>	High	1.3.1	<1.3.2	No	<code>@expo/cli</code> , <code>@expo/code-signing-certificates</code> , <code>selfsigned</code> → <code>node-forge</code>	Used for HTTPS dev server certificates. Not in runtime bundle.
<code>@babel/helpers</code>	Moderate	7.26.9	<7.26.10	No	Babel transpilation	ReDoS in generated code only affects build speed, not runtime. Transpiled output is safe.
<code>js-yaml</code>	Moderate	3.14.1	<3.14.2	No	<code>@expo/xcpretty</code> , <code>cosmicconfig</code> → <code>js-yaml</code>	Used for config parsing at build time. Not in runtime.
<code>@expo/devcert</code>	Low	1.1.4	≤1.1.4	No	<code>@expo/cli</code> → <code>@expo/devcert</code>	Dev HTTPS certificates. Not in runtime.
<code>tmp</code>	Low	0.0.33 / 0.2.3	≤0.2.3	No	<code>@expo/devcert</code> → <code>tmp</code>	Temp file handling for dev certs. Not in runtime.
<code>brace-expansion</code>	Low	2.0.1	≤2.0.1	No	Various build tools: <code>@expo/fingerprint</code> , <code>jest</code> , Metro	ReDoS in glob pattern matching. Build-time only.
<code>compression</code>	Low	1.8.0	≤1.8.0	No	Dev server middleware	HTTP compression for dev server. Not in production.
<code>on-headers</code>	Low	1.0.2	<1.1.0	No	Dev server middleware via <code>compression</code>	Header manipulation in dev server. Not in production.

Package	Severity	Installed Version	Vulnerable Range	Affects Runtime	Source Chain	Notes
min-document	Low	2.19.0	≤2.19.0	Yes	@cardano-sdk/key-management → get-random-values → global → min-document	Prototype pollution. Low severity (CVSS 0). Present in Cardano SDK's polyfill chain.

Summary by Impact Category

Mitigated (Not Applicable to Mobile)

1. `sha.js` (Critical upstream, mitigated)
 - Path: `@cardano-sdk/key-management` → `pbkdf2` → `create-hash / create-hmac` → `sha.js`
 - Impact: Hash rewind vulnerability could allow crafted data to bypass integrity checks
 - Status: Not applicable to `lace-mobile`. Metro redirects `pbkdf2` to `react-native-quick-crypto`, so iOS/Android builds use native crypto and never execute this vulnerable JS code path.
 - Note: Only web builds (not in scope for `lace-mobile`) would be affected.

Affects Mobile Runtime (Low Priority)

2. `min-document` (Low)
 - Path: `@cardano-sdk/key-management` → `get-random-values` → `global` → `min-document`
 - Impact: Low-severity prototype pollution (CVSS 0)
 - Mitigation: No known exploit path in our usage. Monitor for patches.

Development-Only (Conditional Runtime)

3. `jws` (High) - via `@redux-devtools/remote`
 - Path: `@redux-devtools/remote` → `socketcluster-client` → `ag-auth` → `jsonwebtoken` → `jws`
 - Impact: HMAC signature verification bypass
 - Condition: Only included when `ENV === 'development'` (see `initialize-store.ts`)
 - Recommendation: Ensure production builds use `NODE_ENV=production` and that `@redux-devtools/remote` is tree-shaken out. Consider moving to `devDependencies`.

Build-Time Only (No Runtime Impact)

The following vulnerabilities affect build tooling only and are not shipped in the mobile app bundle:

- `form-data` (Critical) - Expo CLI
- `glob` (High) - CLI tools, config parsers
- `image-size` (High) - Metro bundler
- `node-forge` (High) - Dev HTTPS certificates
- `@babel/helpers` (Moderate) - Transpilation
- `js-yaml` (Moderate) - Config parsing
- `@expo/devcert` (Low) - Dev certificates
- `tmp` (Low) - Temp files
- `brace-expansion` (Low) - Glob matching

- `compression` (Low) - Dev server
- `on-headers` (Low) - Dev server

Recommendation

Immediate Actions

1. Move `@redux-devtools/remote` to `devDependencies`
 - This package is only used when `ENV === 'development'`
 - Moving it to `devDependencies` will exclude it from production audits and ensure tree-shaking
 - This removes the `jws` vulnerability from production scope

Already Mitigated (No Action Required)

2. `sha.js` vulnerability is mitigated on mobile
 - Metro config redirects `pbkdf2` → `react-native-quick-crypto` (see `metro.config.js` line 90-95)
 - iOS/Android builds use native crypto implementations that are not affected
 - The vulnerable JS code path is never executed on mobile platforms

Medium-Term Actions

3. Upgrade Expo SDK when available
 - Many build-time vulnerabilities (`form-data`, `glob`, `image-size`, `node-forge`) will be fixed in upcoming Expo SDK versions
 - Track Expo releases for updates to these transitive dependencies
4. Add CI gating:
 - Run `npm audit --workspace lace-mobile --omit=dev --audit-level=high` in CI
 - Fail builds on new `high` / `critical` issues affecting runtime dependencies
 - Consider using `npm audit --json` with a script that filters for runtime-only packages

Low Priority

5. Monitor for patches:
 - `min-document` - Low severity, no immediate action required
 - `brace-expansion` - Low severity ReDoS in build tools
 - `sha.js` - Mitigated on mobile; only relevant for web builds (out of scope)

Notes / Scope

- The scan is scoped to the `lace-mobile` workspace, but `--omit=dev` still includes build tooling that is listed under `dependencies` (notably Expo/Metro tooling). Many advisories are therefore build-time only and not shipped in the final mobile runtime bundle.
- React Native apps use Metro bundler which only includes code that is actually imported at runtime. Build tools like `@expo/cli`, `metro`, and `@babel/*` are not bundled into the app.

- The `metro.config.js` includes polyfill redirects (e.g., `pbkdf2` → `react-native-quick-crypto`) that affect which vulnerable code paths are actually used at runtime.

[L 204] NUMERIC-03: rawToBigInt parseInt Precision Loss

	Level	Severity	Status
	2	Low	Acknowledged

Category: Numeric Safety

Description

The `rawToBigInt` utility function in `format-number.ts` converts a string amount to a `bigint` by first parsing it through JavaScript's `parseInt()`, which returns a `number`. This intermediate conversion to `number` can cause precision loss for values exceeding `Number.MAX_SAFE_INTEGER` (9,007,199,254,740,991).

`packages/lib/util-render/src/format-number.ts`

Vulnerable Code

```
1 // Lines 287-293 typescript
2 export const rawToBigInt = (rawValue: string, decimals: number): bigint => {
3   if (!rawValue || rawValue.trim() === '') return 0n;
4   const normalized = convertAmountToNormalized(rawValue, decimals);
5
6   const integerPart = parseInt(normalized.split('.')[0], 10); // PROBLEM: parseInt
   returns number
7   return BigInt(integerPart || '0'); // Precision already lost before BigInt conversion
8 };
```

The Problem

`parseInt()` returns a JavaScript `number`, which loses precision for integers larger than `MAX_SAFE_INTEGER`:

```
1 parseInt('9007199254740993', 10) // Returns 9007199254740992 (precision javascript
   loss!)
2 BigInt('9007199254740993')      // Returns 9007199254740993n (correct)
```

Proof of Concept

```
1 import { rawToBigInt } from '@lace-lib/util-render'; typescript
2
3 // Value above MAX_SAFE_INTEGER
4 const largeValue = '9007199254740993';
5 const result = rawToBigInt(largeValue, 0);
6
7 console.log(result); // 9007199254740992n (WRONG! Should be 9007199254740993n)
8 console.log(BigInt(largeValue)); // 9007199254740993n (correct)
```

Where This Is Used

The function is used in the send flow for converting user input to BigInt amounts:

```
1 // packages/module/send-flow/src/pages/Send/useSendSheet.ts (lines 326-328) typescript
2 const rawValue = parseLocaleNumber(localeValue || '0');
3 const bigintValue = rawToBigInt(rawValue || '0', decimals);
4 const bigNumberValue = BigNumber(bigIntegerValue);
```

Risk Assessment

Likelihood: Low

- Most crypto amounts don't exceed MAX_SAFE_INTEGER in their smallest unit
- Cardano lovelace: MAX_SAFE_INTEGER = 9,007,199,254 ADA (well above total supply)
- Bitcoin satoshi: MAX_SAFE_INTEGER = 90,071,992 BTC (well above total supply)

Impact: High (if triggered)

- User could send incorrect amount
- Display could show wrong value
- Transaction could have wrong amount

Overall: Low severity due to low likelihood, but the fix is trivial.

Recommendation

Replace `parseInt()` with direct `BigInt()` conversion from string:

```
1 export const rawToBigInt = (rawValue: string, decimals: number): bigint => typescript
2 {
3   if (!rawValue || rawValue.trim() === '') return 0n;
4   const normalized = convertAmountToNormalized(rawValue, decimals);
5   // Split to get integer part
6   const integerPart = normalized.split('.')[0];
7   // Convert directly to BigInt, avoiding number intermediate
8   if (!integerPart || integerPart === '' || integerPart === '-') {
9     return 0n;
10  }
11  try {
12    return BigInt(integerPart);
13  } catch {
14    return 0n;
15  }
```

```
17     }
18 };
```

Alternative: Validate Input Range

If you want to explicitly reject values that would cause precision loss:

```
1 export const rawToBigInt = (rawValue: string, decimals: number): bigint =>
  {
2     if (!rawValue || rawValue.trim() === '') return 0n;
3     const normalized = convertAmountToNormalized(rawValue, decimals);
4     const integerPart = normalized.split('.')[0];
5
6     if (!integerPart || integerPart === '') return 0n;
7
8     // Direct BigInt conversion (safe for any size)
9     return BigInt(integerPart);
10  };
```

Test Coverage

Add test cases for large values:

[illegible]

Related Findings

- NUMERIC-01: Bitcoin satoshi calculations use JavaScript number type
- NUMERIC-02: Number/BigInt Mixing in Bitcoin Input Selection

[L 205] BUILD-05: API Keys Bundled in Client Application

	Level	Severity	Status
	2	Low	Acknowledged

Category: Build Configuration / Secrets Management

Affected Files:

- `apps/lace-mobile/src/app/util/config.ts`
- `apps/lace-mobile/.env.development`
- `apps/lace-mobile/.env.production`

Description

Third-party API keys and tokens are bundled directly into the application via environment variables with the `EXP0_PUBLIC_` prefix. These become part of the JavaScript bundle and are extractable from the compiled APK/IPA:

- `EXP0_PUBLIC_POSTHOG_API_TOKEN` — Analytics token
- `EXP0_PUBLIC_BLOCKFROST_PROJECT_ID_*` — Blockchain API keys (mainnet, preprod, preview)
- `EXP0_PUBLIC_MAESTRO_PROJECT_ID_*` — Bitcoin API keys
- `EXP0_PUBLIC_SENTRY_DSN` — Error reporting endpoint

These keys can be extracted by:

- Decompiling the APK/IPA
- Searching strings in the binary
- Intercepting network traffic

For Expo/React Native apps this is generally expected: anything under `EXP0_PUBLIC_*` should be treated as a public client identifier, not a secret.

Recommendation

1. Accept the risk for read-only public APIs — Blockfrost and Maestro keys for public blockchain data may be acceptable to bundle if:
 - Rate limits are enforced server-side
 - Keys can be rotated easily
 - Abuse is monitored
2. For sensitive keys, consider:
 - Proxying requests through a backend service that holds the keys
 - Using token exchange patterns where the app authenticates and receives short-lived tokens
 - Implementing request signing to detect abuse
3. Operational controls:
 - Monitor API usage for anomalies
 - Set up alerts for quota abuse
 - Ensure keys can be rotated without app updates (via remote config or forced updates)
 - Use separate keys for production vs development

4. Sentry DSN specifically:

- DSNs are designed to be public but enable rate limiting
- Configure Sentry project settings to filter/rate-limit if abuse occurs

[L 206] BUILD-07: iOS Export Compliance Declaration May Be Inaccurate

Level	Severity	Status
2	Low	Acknowledged

Category: Build Configuration / Compliance

Affected File: apps/lace-mobile/app.json

Description

The app declares `ITSApUsesNonExemptEncryption: false` in the iOS Info.plist configuration:

```

1 {
2   "ios": {
3     "infoPlist": {
4       "ITSApUsesNonExemptEncryption": false
5     }
6   }
7 }
```

This declaration indicates the app does not use encryption that requires export compliance documentation. However, the app uses several cryptographic functions:

- EMIP-3 encryption/decryption — Used for mnemonic/recovery phrase protection
- Blake2b hashing — Via native Apollo module
- Ed25519 signatures — For transaction signing
- BIP32 key derivation — For HD wallet address generation

The accuracy of this declaration depends on whether these cryptographic uses qualify for exemptions under U.S. export regulations (typically authentication, digital signatures, and protecting stored data have exemptions).

Recommendation

This is primarily a compliance / App Store review risk, not a software vulnerability.

1. Conduct export compliance review to determine if the current declaration is accurate:
 - Review BIS (Bureau of Industry and Security) exemption categories
 - Common exemptions: authentication, access control, digital signatures, stored data protection
 - Wallet encryption typically qualifies for exemption but should be formally reviewed
2. If exempt, document the basis:
 - Create an internal compliance document explaining why the exemption applies
 - Keep records for potential App Store review questions
3. If NOT exempt:

- File for an Encryption Registration (ERN) with BIS
- Update the plist to `ITSAAppUsesNonExemptEncryption: true`
- Be prepared to provide encryption documentation during App Store review

[L 207] BUILD-08: Dev Artifacts Exclusion from Release Builds Needs Verification

	Level	Severity	Status
	2	Low	Acknowledged

Category: Build Configuration / Release Hygiene

Affected File: `apps/lace-mobile/src/app/util/all-modules.ts`

Affected Components: Build pipeline

Description

Development-only code and artifacts may be included in production release builds if not properly excluded. Potential concerns include:

1. Dev modules present in the bundle (`@lace-module/feature-dev` and `@lace-module/test-api`) are imported in `all-modules.ts`, so Metro will include them in the JS bundle even if feature flags prevent them from being selected at runtime.
2. Storybook components: Bundling is driven by imports, not whether a package is listed in `dependencies` vs `devDependencies`. If storybook code is imported by production entry-points, it will be bundled.
3. Test fixtures and mock data: Any hardcoded test mnemonics or passphrases
4. Debug logging statements: Console logs that may leak information
5. Feature flag override mechanism: `metro.config.js` checks for `feature-flags.override.ts` at build time

The Metro bundler's tree-shaking and module resolution behavior determines what ends up in the final bundle.

Recommendation

1. Verify bundle contents by analyzing the production build:

```
1 # Build production bundle
2 npx expo export --platform ios
3
4 # Search for dev indicators
5 grep -r "storybook\|test-api\|feature-dev\|addWallet\|setFeatureFlags" dist/
```

2. Use bundle analysis tools:

```
1 # Install and run react-native-bundle-visualizer or expo-atlas
2 npx expo-atlas
```

3. Ensure dev modules are import-guarded:

```
1 // Only import in development
2 const devModules = __DEV__ ? require('./dev-modules') : [];
```

typescript

4. Add CI verification:

- Build production bundle
- Search for known dev-only strings
- Fail build if found

5. Review metro.config.js to ensure the feature-flags override mechanism cannot accidentally be triggered in production:

- Verify `.gitignore` includes `feature-flags.override.ts`
- Add CI check that the override file does not exist

6. Treat this as defense-in-depth and fix root causes:

- Ensure `FEATURES_DEV` / `TEST_API` are not enabled by default in production (see BUILD-01)
- Ensure `feature-flags.override.ts` cannot be used in production builds (see BUILD-09)

[L 208] BUILD-09: Feature Flag Override Mechanism in Production Builds

	Level	Severity	Status
	2	Low	Acknowledged

Category: Build Configuration / Release Hardening

Affected File: apps/lace-mobile/metro.config.js

Description

The Metro bundler configuration includes logic to substitute the default feature flags file with an override file at build time:

```

1  if (moduleName.includes('feature-flags') || moduleName.endsWith('feature-
   flags')) {
2      const overridePath = path.resolve(projectRoot, 'src', 'app', 'feature-
   flags.override.ts');
3      if (fs.existsSync(overridePath)) {
4          return { type: 'sourceFile', filePath: overridePath };
5      }
6  }
```

javascript

While `feature-flags.override.ts` is gitignored, this mechanism exists in the production Metro configuration. Risks include:

1. Accidental inclusion: Developer creates override file, forgets to delete before release
2. CI environment manipulation: If CI runner is compromised, an override file could be injected
3. Build artifact tampering: Anyone with write access to the build directory could inject flags

The override completely replaces the default flags, potentially enabling debug modules or disabling security features.

Recommendation

This is primarily a build-hardening concern: if an attacker can write files into the build workspace they can usually do more than just add an override file. The main risk is accidental inclusion during a local/manual release build or cache poisoning in CI.

1. Remove or strictly guard the override mechanism in production builds:
 - Only allow override when `process.env.NODE_ENV === 'development'` (or an explicit dev-only env var)
 - Log a prominent warning when an override is active
2. Add CI verification that the override file does not exist:

```
1 if [ -f "src/app/feature-flags.override.ts" ]; then
2   echo "ERROR: feature-flags.override.ts must not exist in CI builds"
3   exit 1
4 fi
```

bash

3. Use environment-specific flag files instead of an override mechanism:
 - `feature-flags.production.ts`
 - `feature-flags.development.ts`
 - Select at build time via environment variable
4. If override is needed for development:
 - Move the check inside a `process.env.NODE_ENV === 'development'` guard
 - Never allow override in CI/production environments
5. Remember: `.gitignore` prevents commits, but it does not prevent the file from existing on disk. CI must enforce absence explicitly.

[I 101] Native Apollo Sync API Inconsistencies (Latent Risk)

Level	Severity	Status
1	Informational	Acknowledged

Affected Files:

- apps/lace-mobile/apollo-ios/Apollo/ApolloModule.swift
- apps/lace-mobile/apollo-android/ApolloModule.kt

Description

The native Apollo cryptography module exposes synchronous API methods that are either non-functional stubs or semantically inconsistent across iOS and Android platforms. While these sync APIs are currently unused in production code, their presence represents a latent risk for future development.

Issue 1: Android `blake2bHashSync` returns empty string (stub implementation)

The Android implementation has a stub sync method that always returns an empty string instead of computing the actual hash:

```

1 // ApolloModule.kt - Lines 78-81
2 @Throws(Exception::class)
3 private fun blake2bHexSync(inputHex: String, outLen: Int): String {
4     return "" // STUB: Returns empty string instead of hash
5 }
6
7 // Lines 91-96
8 fun blake2bHashSync(inputHex: String, outLen: Int): String {
9     return try {
10         blake2bHexSync(inputHex, outLen) // Calls the stub
11     } catch (e: Exception) {
12         ""
13     }
14 }
```

This is particularly dangerous because:

- The method silently returns invalid data instead of throwing an error
- A developer might use this sync API expecting valid cryptographic output
- Hash verification or integrity checks would silently fail

Issue 2: iOS `derivePublicKeySync` uses seed-based derivation (semantic mismatch)

The iOS sync method `derivePublicKeySync` calls `deriveKeyHexSync` which performs seed-based private key derivation, not public key derivation from an extended public key:

```

1 // ApolloModule.swift - Lines 191-202
2 @objc(derivePublicKeySync:role:index:)
```

```

3  static func derivePublicKeySync(_ pubKeyHex: String,
4                                  role: NSNumber,
5                                  index: NSNumber) -> String {
6      do {
7          return try deriveKeyHexSync(pubKeyHex, // Calls seed-based derivation
8                                      role: role.int32Value,
9                                      index: index.int32Value)
10     } catch {
11         return ""
12     }
13 }
14
15 // deriveKeyHexSync (Lines 87-147) does seed-based derivation:
16 static func deriveKeyHexSync(_ pubKeyHex: String, role: Int32, index: Int32) throws ->
String {
17     // ...
18     let masterKey: EdHDKKey
19     masterKey = EdHDKKey.Companion.shared.doInitFromSeed(seed: seedArr) // SEED
derivation!
20     // ...
21     guard let privArr = childKey.privateKey as KotlinByteArray? else { ... } // Returns
private key material
22     // ...
23 }

```

This is problematic because:

- The method name suggests public key derivation from an xpub
- It actually expects a seed and derives through private keys
- Private key material is handled in the sync call path
- Inconsistent with the async `derivePublicKey` which correctly uses `deriveKeyHexSync2` for xpub-based derivation

Platform comparison

Method	iOS	Android
<code>derivePublicKey</code> (async)	Correct (xpub-based)	Correct (xpub-based)
<code>derivePublicKeySync</code>	Seed-based (wrong semantics)	Correct (xpub-based)
<code>blake2bHash</code> (async)	Correct	Correct
<code>blake2bHashSync</code>	Correct	Stub (returns "")

Risk assessment

- Current risk: Low — static analysis confirms no JS callsites use these sync APIs
- Future risk: Medium — developers may incorrectly assume sync APIs work identically to async versions
- Crypto safety: Potential for silent failures or incorrect key derivation if sync APIs are used

Recommendation

1. Remove unused sync APIs entirely
 - Since they are not exposed to React Native and not used anywhere, the safest option is to delete them
 - This eliminates the risk of future misuse
2. If sync APIs must be kept, fix them:

Android — Implement real blake2bHashSync:

```
1 private fun blake2bHexSync(inputHex: String, outLen: Int): String {
2     val inputBytes = inputHex.decodeHex()
3     val hashBytes = computeBlake2b(inputBytes, outLen)
4     return hashBytes.joinToString("") { "%02x".format(it) }
5 }
```

kotlin

iOS — Fix derivePublicKeySync to use correct derivation:

```
1 @objc(derivePublicKeySync:role:index:)
2 static func derivePublicKeySync(_ pubKeyHex: String,
3                                 role: NSNumber,
4                                 index: NSNumber) -> String {
5     do {
6         // Use deriveKeyHexSync2 (xpub-based), not deriveKeyHexSync (seed-based)
7         return try deriveKeyHexSync2(pubKeyHex,
8                                       role: role.int32Value,
9                                       index: index.int32Value)
10    } catch {
11        return ""
12    }
13 }
```

swift

3. Add documentation/deprecation warnings
 - If keeping sync APIs, add clear documentation about their behavior
 - Consider marking them as deprecated to discourage use

4. Add unit tests for platform parity
 - Ensure iOS and Android produce identical outputs for identical inputs
 - Test both async and sync variants with the same test vectors

Verification

To confirm these APIs are unused, run:

```
1 # Search for sync API usage in JS/TS code bash  
2 grep -r "derivePublicKeySync\|blake2bHashSync" packages/ apps/
```

Current status: Confirmed unused — only the async `derivePublicKey` and `blake2bHash` methods are called from `packages/module/crypto-apollo/src/`.

[I 102] Non-functional Address Book

	Level	Severity	Status
	1	Informational	Acknowledged

Category: User Experience & Safety / Transaction Safety Nets

Affected Components: Send flow, address book, recent recipients, transaction review UI

Description

The wallet exposes an Address Book entry point in the send flow, but it does not provide a functional Contacts capability. Users cannot manage saved recipients (add/edit/delete/view), persist trusted recipients, or receive meaningful safety signals during recipient selection. This forces repeated reliance on error-prone copy/paste or ad-hoc QR scans for address entry, increasing the risk of mis-sends, clipboard tampering, and exposure to address-poisoning scams.

These gaps do not constitute a direct exploit, but they increase the likelihood of user-induced loss.

Observations

- While there is an address book field in the send flow, there is no way to add/edit/delete addresses in the address book.
- There is no way to view saved addresses, and no access point to the address book from the main menu.
- There is no way to save a recently used address to the address book.
- There is no import/export for address book entries.
- There is no way to tag addresses (e.g., watch-only) or add notes.
- There is no warning when sending to an address that is not in the address book.
- There is no way to mark addresses as risky or maintain a blocklist.

Recommendation

1. Implement a functional Contacts / Address Book

- Provide CRUD for contacts: `add / edit / delete / view` with name + address (minimum), plus optional notes.
- Add `Save recipient` from recent recipients and from the transaction review step.
- Ensure contacts are accessible from the Dashboard/Menu/Settings (not only from the send flow).
- (Optional) Support `import / export` (encrypted) of contacts.
- (Optional) Support watch-only tags for addresses.
- IMPORTANT:
 - Ensure that contacts are chain-specific (e.g., Cardano vs. Bitcoin) to prevent cross-chain address confusion.

- Ensure that contacts are network-specific (e.g., mainnet vs. testnet/preview/preprod) to prevent sending test assets to mainnet addresses or vice versa.

2. Add lightweight recipient safety nets

- In the recipient field, suggest matching contacts and show clear identity cues (label/alias).
- At review time, indicate whether the recipient is a saved contact and warn when sending to an unknown/unsaved address.
- Consider optional Scam/Blocklist flags that users can maintain for addresses they distrust.

[I 103] Missing Balance Privacy Feature

	Level	Severity	Status
	1	Informational	Acknowledged

Category: Privacy / UI

Affected Components: Portfolio/dashboard, account list, main app UI

Description

The app does not provide a Hide Balance / Privacy Mode option to mask sensitive values (portfolio total and/or per-asset balances) in the main UI. In practice, users often open wallets in public, screen-share during support, or capture screenshots—without intending to disclose holdings. A simple masking mode reduces accidental exposure and discourages opportunistic targeting (e.g., shoulder-surfing, social engineering).

This is not a direct exploit, but it is a widely adopted privacy safeguard in other wallets and improves safe day-to-day usage.

Observations

- No hide-balance / privacy toggle is available in the main app UI.

Recommendation

- Add a Privacy Mode toggle (e.g., “Hide balances”) that masks:
 - portfolio total
 - per-asset balances (and optionally fiat equivalents)
- Provide a quick toggle (eye icon / tap-and-hold) on the portfolio header and persist the setting per device/session.
- Consider optional UX hardening:
 - “tap to reveal” momentary reveal
 - auto-revert after a short time or on app background
 - mask balances in account list and during screen-share contexts where feasible

[I 104] Transaction and Wallet Transfer via QR and File Import

	Level	Severity	Status
	1	Informational	Acknowledged

Category: User Experience & Safety

Affected Components: Receive flow, Send flow, wallet import/export UX

Description

The wallet supports QR usage for basic address sharing, but does not provide two common quality-of-life capabilities seen in many wallets:

1. QR-based transaction requests (encode recipient + optional amount/metadata) to reduce manual entry and mis-sends.
2. Advanced import/export options for wallet management, such as QR or file-based encrypted seed-phrase import/export.

These features can enhance usability and reduce user error in routine payments, while also improving wallet management convenience.

Note: Seed phrase QR/file import/export is particularly sensitive and should be treated as an advanced feature with strong guardrails, since it can normalize digital handling of the seed phrase and introduce new leak paths.

Observations

- Currently, the wallet's receive QR code encodes only the receiving address. There is no option to generate a transaction request QR that encodes the receiving address plus optional amount and note/metadata. This feature would allow senders to scan the QR code (or paste the encoded data/URI) and be taken directly to a pre-constructed transaction review screen (ready to sign after verification), rather than manually entering details, reducing entry errors and improving UX.
- Currently, the wallet only supports importing wallets by manually entering the seed phrase. There are no advanced import/export mechanisms such as QR-based or file-based encrypted seed phrase import/export.

Recommendation

1. Add QR-based transaction requests
 - In the Receive flow, allow generating a QR containing:
 - receiving address
 - optional amount
 - optional note/metadata
 - Allow sharing/exporting this encoded transaction request via on-screen display, clipboard copy, or via other apps (email, messaging, etc).

- In the Send flow, support scanning, importing, or pasting this encoded transaction request.
- Display all transaction details (recipient, amount, fees, note) for sender review before signing.
- Ensure these shared encoded transaction requests are chain and network specific.

2. Implement advanced wallet import/export options with strong guardrails

- In the `Add Wallet` flow, `Import existing wallet` section, add options for:
 - QR code scanning of an encrypted seed phrase
 - File import of an encrypted seed phrase
- In the wallet settings, add an `Export Wallet` section:
 - Exporting the encrypted seed phrase as a QR code (with strong warnings)
 - Exporting the encrypted seed phrase as a file (with strong warnings)
 - Export via: on-screen QR code display, downloadable file, or sharing via other apps (email, messaging, etc)
- Secure these features with the following precautions:
 - Require this feature to be disabled by default; users must explicitly enable it in settings.
 - Treat this feature as Advanced with explicit warnings and confirmations when enabling.
 - Display an educational prompt about the risks and best practices during import/export.
 - Require re-authentication before any export.
 - Use passphrase-based authenticated encryption, and require a strong passphrase.
 - Do not allow unencrypted seed phrase export.
 - Provide clear instructions on securely handling exported seed phrases.
 - Reduce exposure during export (no screenshots, auto-hide, timeouts).
 - Recommend users to delete exported files/QRs after use.

[I 105] Sensitive Cryptographic Material Exposure Risk

Level	Severity	Status
1	Informational	Acknowledged

Category: Observability / Data Redaction

Affected Files:

- `packages/lib/observability/src/ObservableLogger.ts`
- `packages/contract/tx-executor/src/store/generic-error-results.ts`
- `packages/module/blockchain-bitcoin/src/tx-executor-implementation/confirm-tx.ts`
- `packages/contract/authentication-prompt/src/value-objects/auth-secret.ts`
- `packages/contract/recovery-phrase/src/store/slice.ts`

Description

The application handles sensitive cryptographic material (authentication secrets, recovery phrases, private keys) without systematic protection mechanisms against accidental exposure to telemetry services (PostHog, Sentry). Code analysis identified multiple pathways where sensitive data could be serialized and transmitted during logging, error handling, or analytics tracking.

Key Observations:

1. `ObservableLogger.formatMessage()` at line 84 uses `JSON.stringify(p)` to serialize objects without filtering. Logger methods forward output to Sentry breadcrumbs and messages, potentially exposing any object passed to logging functions.
2. Transaction signing error handling at `confirm-tx.ts:129-130` passes caught exceptions to `genericErrorResults.confirmTx({ error })`, which calls `serializeError(error)` using the npm package `serialize-error` 8.1.0. This serializes error stack traces that may contain local variables holding private keys (`rootPrivateKey`, `rootKeyPair`, `keyPair`, `tweakedPrivateKey`, `signers`). The serialized error is stored in Redux state.
3. Recovery phrase stored in Redux state at `slice.ts:13` as `mnemonic: Serializable<ByteArray[]> | null` without wrapper protection. Accessible through Redux DevTools and state exports.
4. `AuthSecret` is a Tagged type alias without `toString()`, `toJSON()`, or `util.inspect.custom` overrides. Standard JavaScript serialization exposes the underlying `ByteArray`.
5. Analytics payloads accept `Record<string, JsonType>` without filtering for sensitive data represented as primitives (hex strings, base64, numeric arrays).

Existing Protections:

- PostHog constrains payloads to `JsonType`
- Sentry configured with `sendDefaultPii: false`
- Authentication prompt clears secrets after 10 seconds
- BitcoinSigner implements `clearSecrets()` method

- Root private key cleanup in finally block

These protections do not prevent serialization during normal operation or in error conditions.

Exposure Scenarios:

- Developer adds `logger.error('context:', { authSecret })` during debugging
- Exception during transaction signing serializes error with private keys in scope
- Redux state export for support includes recovery phrase
- Analytics event includes encrypted/encoded sensitive data

Recommendation

Implement a Redacted wrapper type following the Effect-TS pattern to prevent accidental serialization.

Core Implementation:

Create `packages/lib/util-redacted`:

```
1  const RedactedSymbol = Symbol.for('@lace/Redacted');
2
3  export interface Redacted<out A = string> {
4    readonly _tag: 'Redacted';
5    readonly [RedactedSymbol]: A;
6  }
7
8  export const make = <A>(value: A): Redacted<A> => ({
9    _tag: 'Redacted' as const,
10   [RedactedSymbol]: value,
11   toString: () => '[REDACTED]',
12   toJSON: () => '[REDACTED]',
13   [Symbol.for('nodejs.util.inspect.custom')]: () => '[REDACTED]',
14 } as Redacted<A>);
15
16 export const value = <A>(self: Redacted<A>): A => self[RedactedSymbol];
17
18 export const unsafeWipe = <A>(self: Redacted<A>): void => {
19   const val = self[RedactedSymbol];
20   if (val instanceof Uint8Array || Buffer.isBuffer(val)) {
21     val.fill(0);
22   }
23   delete (self as any)[RedactedSymbol];
24 };
```

```

25
26 export const isRedacted = (value: unknown): value is Redacted<unknown> =>
27   typeof value === 'object' && value !== null && '_tag' in value && value._tag ===
   'Redacted';
28
29 export const filterRedacted = (obj: unknown): unknown => {
30   if (isRedacted(obj)) return '[REDACTED]';
31   if (typeof obj !== 'object' || obj === null) return obj;
32   if (Array.isArray(obj)) return obj.map(filterRedacted);
33   const filtered: Record<string, unknown> = {};
34   for (const key in obj) filtered[key] = filterRedacted((obj as any)[key]);
35   return filtered;
36 };

```

Apply Protection:

1. Update `ObservableLogger.formatMessage()` line 84:

```
1 return JSON.stringify(filterRedacted(p));
```

typescript

2. Update `SentryProvider.captureException()` and `addBreadcrumb()`:

```

1 scope.setExtra(key, filterRedacted(value));
2 data: breadcrumb.data ? filterRedacted(breadcrumb.data) : undefined

```

typescript

3. Wrap `AuthSecret`:

```

1 export type AuthSecret = Redacted<Tagged<ByteArray, 'AuthSecret'>>;
2 export const AuthSecret = (authSecret: ByteArray): AuthSecret =>
3   Redacted.make(authSecret as Tagged<ByteArray, 'AuthSecret'>);

```

typescript

4. Wrap recovery phrase:

```
1 mnemonic: Redacted<Serializable<ByteArray[]>> | null;
```

typescript

5. Wrap transaction signing keys:

```

1 let rootPrivateKey: Redacted<Buffer> | null = null;
2 rootPrivateKey = Redacted.make(Buffer.from(decryptedBytes));
3 // Unwrap only at cryptographic operations:
4 seed: Redacted.value(rootPrivateKey)

```

typescript

References

- [Effect-TS Redacted Implementation](#)

End of Report



No Witness Labs — Confidential

Copyright © 2025